

These are the slides for the Beginning C++ Programming - From Beginner to Beyond on Udemy. They are provided free of charge to all students.

More information about the course: <https://lpa.dev/u1bcppp>

If you have any questions or queries, please add your feedback in the Q&A section of the course on Udemy.

Best regards,

Tim Buchalka
Learn Programming Academy

Beginning C++ Programming Slides

Main Course Slides.

Welcome and Introduction to the Course

- About me
- My assumptions
- Your background
- The curriculum and Modern C++
- Practice!
- Please ask questions

Why Learn C++?

- Popular
 - Lots of code is still written in C++
 - Programming language popularity indexes
 - Active community, GitHub, stack overflow
- Relevant
 - Windows, Linux, Mac OSX, Photoshop, Illustrator, MySQL, MongoDB, Game engines, more ...
 - Amazon, Apple, Microsoft, PayPal, Google, Facebook, MySQL, Oracle, HP, IBM, more...
 - VR, Unreal Engine, Machine learning, Networking & Telecom, more...
- Powerful
 - fast, flexible, scalable, portable
 - Procedural and Object-oriented
- Good career opportunities
 - C++ skills always in demand
 - C++ = Salary++

Modern C++ and the C++ Standard

- Early 1970s
 - C Programming Language
 - Dennis Ritchie
- 1979
 - Bjarne Stroustrup
 - C with Classes
- 1983
 - Name changed to C++
- 1989
 - First commercial release
- 1998
 - C++98 Standard
- 2003
 - C++03 Standard
- **2011**
 - **C++11 Standard**
- **2014**
 - **C++14 Standard**
- **2017**
 - **C++17 Standard**

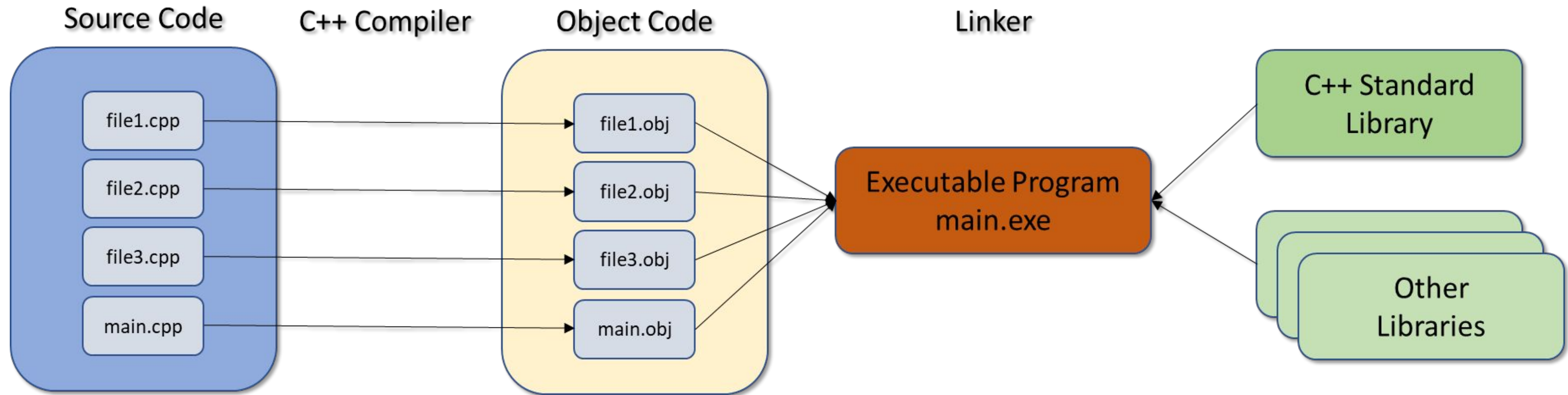
Modern C++ and the C++ Standard

- Classical C++
 - Pre C++11 Standard
- Modern C++
 - C++11
 - Lots of new features
 - C++14
 - Smaller changes
 - C++17
 - Simplification
 - Best practices
 - Core Guidelines

How does it all work?

- You must tell the computer EXACTLY what to do
 - Program – like a recipe
- Programming language
 - source code
 - high-level
 - for humans
- Editor – used to enter program text
 - .cpp and .h files
- Binary or other low-level representation
 - object code
 - for computers
- Compiler – translates from high-level to low-level
- Linker – links together our code with other libraries
 - Creates executable program
- Testing and Debugging – finding and fixing program errors

The C++ Build Process



Integrated Development Environments (IDEs)

- Editor
 - Compiler
 - Linker
 - Debugger
 - Keep everything in sync
- CodeLite
 - Code::Blocks
 - NetBeans
 - Eclipse
 - CLion
 - Dev-C++
 - KDevelop
 - Visual Studio
 - Xcode

Section Overview

- Microsoft Windows, Mac OSX, Ubuntu Linux 18.04
 - C++ Compiler
 - CodeLite Integrated Development Environment (IDE)
 - Configure CodeLite
 - Create a Default CodeLite Project Template
- Using the Command-line
- Using a Web-based compiler



Please check the **Resources** for the videos as I will post updates there as needed.

Using the command-line interface

- A text editor (not a Word Processor)
 - A command-prompt or terminal window
 - An installed C++ compiler
-
- No need for an IDE
 - Simple, efficient workflow
 - Better as you gain experience
 - Can be used if you are overwhelmed by IDEs
 - Useful if hardware resources are limited

Curriculum Overview

- Getting Started
- Structure of a C++ Program
- Variables and Constants
- Arrays and Vectors
- Strings in C++
- Expressions, Statements and Operators
- Statements and Operators
- Determining Control Flow
- Functions
- Pointers and References
- OOP – Classes and Objects
- Operator Overloading
- Inheritance
- Polymorphism
- Smart Pointers
- The Standard Template Library (STL)
- I/O Streams
- Exception Handling

Curriculum Overview

Challenge Exercises

- At the end of most course sections
- Develop real C++ programs using what we discussed in the section
- Section challenges
 - Description
 - Starting project
 - Completed solution
- Have fun and keep coding!

Curriculum Overview

Quizzes

- At the end of most sections
- Reinforce the concepts learned in each section
- Quiz style
 - Multiple choice
 - Fill in blank
 - Concept oriented vs. code oriented

Section Overview

- CodeLite IDE Quick Tour
- Our first program
 - Building
 - Running
 - Errors
 - Warnings

Compiler Errors

- Programming languages have rules
- Syntax errors – something wrong with the structure

```
std::cout << "Errors" << std::endl;
```

```
return 0
```

- Semantic errors – something wrong with the meaning
`a + b;` When it doesn't make sense to add `a` and `b`

Compiler Warnings

Do NOT ignore them!

- The compiler has recognized an issue with your code that could lead to a potential problem
- It's only a warning because the compiler is still able to generate correct machine code

```
int miles_driven;  
std::cout << miles_driven;
```

warning: 'miles_driven' is used uninitialized ...

Linker Errors

- The linker is having trouble linking all the object files together to create an executable
- Usually there is a library or object file that is missing

Runtime Errors

- Errors that occur when the program is executing
- Some typical runtime errors
 - Divide by zero
 - File not found
 - Out of memory

Can cause your program to 'crash'

Exception Handling can help deal with runtime errors

Logic Errors

- Errors or bugs in your code that cause your program to run incorrectly
- Logic errors are mistakes made by the programmer

Suppose we have a program that determines if a person can vote in an election and you must be 18 years or older to vote.

```
if (age > 18) {  
    std::cout << "Yes, you can vote!";  
}
```

Test your code!!

Section Overview

The Structure of a C++ Program

- Basic Components
- Preprocessor Directives
- The main function
- Namespaces
- Comments
- Basic I/O

The Structure of a C++ Program

Components of a C++ Program

- Keywords
- Identifiers
- Operators
- Punctuation
- Syntax

Keywords

- Have special meaning in C++
- Are reserved by the C++ language

```
#include <iostream>

int main() {

    int favorite_number;

    std::cout << "Enter your favorite number between 1 and 100: ";
    std::cin >> favorite_number;

    std::cout << "Amazing!! That's my favorite number too!" << std::endl;

    std::cout << "No really!!, " << favorite_number << " is my favorite number!" <<std::endl;

    return 0;

}
```

Identifiers

- Programmer-defined names
- Not part of the C++ language
- Used to name variables, functions, etc.

```
#include <iostream>

int main() {

    int favorite_number;

    std::cout << "Enter your favorite number between 1 and 100: ";
    std::cin >> favorite_number;

    std::cout << "Amazing!! That's my favorite number too!" << std::endl;

    std::cout << "No really!!, " << favorite_number << " is my favorite number!" << std::endl;

    return 0;
}
```

Operators

- Arithmetic operators, assignment, <<, >>
- Are reserved by the C++ language

```
#include <iostream>

int main() {

    int favorite_number;

    std::cout << "Enter your favorite number between 1 and 100: ";
    std::cin >> favorite_number;

    std::cout << "Amazing!! That's my favorite number too!" << std::endl;

    std::cout << "No really!!, " << favorite_number << " is my favorite number!" <<std::endl;

    return 0;

}
```


Punctuation

- Special characters that separate, terminate items

```
#include <iostream>

int main() {

    int favorite_number;

    std::cout << "Enter your favorite number between 1 and 100: ";
    std::cin >> favorite_number;

    std::cout << "Amazing!! That's my favorite number too!" << std::endl;

    std::cout << "No really!!, " << favorite_number << " is my favorite number!" <<std::endl;

    return 0;
}
```

Syntax

- How the programming elements are put together to form a program
- Programming languages have rules

```
#include <iostream>

int main() {

    int favorite_number;

    std::cout << "Enter your favorite number between 1 and 100: ";
    std::cin >> favorite_number;

    std::cout << "Amazing!! That's my favorite number too!" << std::endl;

    std::cout << "No really!!, " << favorite_number << " is my favorite number!" <<std::endl;

    return 0;

}
```

Our Section 4 Challenge Solution

Modified slightly

- Added comments
- Using namespace instead of std::

```
// Preprocessor directive that include the iostream library headers
#include <iostream>

// use the std namespace
using namespace std;

/* Start of the program
 * program execution always begins with main()
 */
int main() {

    int favorite_number;    // declare my favorite number variable

    // Note that I'm no longer using std::
    // Prompt the user to enter their favorite number

    cout << "Enter your favorite number between 1 and 100: ";

    // read the user's input into the variable favorite_number
```


Our Section 4 Challenge Solution

Modified slightly

```
// Preprocessor directive that include the iostream library headers
#include <iostream>

// use the std namespace
using namespace std;

/* Start of the program
 * program execution always begins with main()
 */
int main() {

    int favorite_number;    // declare my favorite number variable

    // Note that I'm no longer using std::
    // Prompt the user to enter their favorite number

    cout << "Enter your favorite number between 1 and 100: ";

    // read the user's input into the variable favorite_number
    cin >> favorite_number;

    // Out put the results to the user
    cout << "Amazing!! That's my favorite number too!" << endl;
    cout << "No really!!, " << favorite_number << " is my favorite number!" << endl;

    return 0;
}
```

Preprocessor Directives

- What is a preprocessor?
- What does it do?
- Directives start with '#'
- Commands to the preprocessor

```
#include <iostream>
#include "myfile.h"
```

```
#if
#elif
#else
#endif
```

Comments

- Ignored by the compiler
- Used to explain your code
- Two styles

```
int favorite_number; // This will store my favorite number

// The following lines convert Euros to US Dollars

/* This is a comment
   that spans multiple lines.
   All of these lines will be ignored by the compiler
*/

// Why should we have to explain our own code?
```

Functions

- `main()` is a required function in C++
- Break up your code into units of functionality
- Can optionally receive and return information

```
/* This is a function that expects two integers a and b
   It calculates the sum of a and b and returns it to the caller
   Note that we specify that the function returns an int
*/

int add_numbers(int a, int b)
{
    return a + b;
}

// I can call the function and use the value that is returns

cout << add_numbers(20, 40);
```

Preprocessor Directives

- What is a preprocessor?
- What does it do?
- Directives start with '#'
- Commands to the preprocessor

```
#include <iostream>
#include "myfile.h"
```

```
#if
#elif
#else
#endif
```

```
#ifdef
#ifndef
#define
#undef
```

```
#line
#error
#pragma
```


The main() function

- Every C++ program must have exactly 1 main() function
- Starting point of program execution
- return 0 indicates successful program execution
- 2 versions that are both valid

```
int main()
{
    // code

    return 0;
}

program.exe

int main(int argc, char *argv[])
{
    // code

    return 0;
}

program.exe argument1 argument2
```

Namespaces

- Why `std::cout` and not just `cout`?
- What is a naming conflict?
- Names given to parts of code to help reduce naming conflicts
- `std` is the name for the C++ 'standard' namespace
- Third-party frameworks will have their own namespaces
- Scope resolution operator `::`
- How can we use these namespaces?

Explicitly using namespaces

```
#include <iostream>

int main()
{
    int favorite_number;

    std::cout << "Enter your favorite number between 1 and 100: ";

    std::cin >> favorite_number;

    std::cout << "Amazing!! That's my favorite number too!" << std::endl;
    std::cout << "No really!!, " << favorite_number
                << " is my favorite number!" << std::endl;

    return 0;
}
```


The using namespace directive

```
#include <iostream>

using namespace std;          // Use the entire std namespace

int main()
{
    int favorite_number;

    cout << "Enter your favorite number between 1 and 100: ";

    cin >> favorite_number;

    cout << "Amazing!! That's my favorite number too!" << endl;
    cout << "No really!!, " << favorite_number
         << " is my favorite number!" << endl;

    return 0;
}
```

Qualified using namespace variant

```
#include <iostream>

using std::cout; // use only what you need
using std::cin;
using std::endl;

int main()
{
    int favorite_number;

    cout << "Enter your favorite number between 1 and 100: ";

    cin >> favorite_number;

    cout << "Amazing!! That's my favorite number too!" << endl;
    cout << "No really!!, " << favorite_number
         << " is my favorite number!" << endl;

    return 0;
}
```

Basic I/O using `cin` and `cout`

`cout`, `cin`, `cerr`, and `clog` are objects representing streams

`cout`

- standard output stream
- console

`cin`

- standard input stream
- keyboard

`<<`

- Insertion operator
- output streams

`>>`

- extraction operator
- input streams

cout and <<

- Insert the data into the cout stream

```
cout << data;
```

- Can be chained

```
cout << "data 1 is " << data1;
```

- Does not automatically add line breaks

```
cout << "data 1 is " << data1 << endl;
```

```
cout << "data 1 is " << data1 << "\n";
```

cin and >>

- Extract data from the cin stream based on data's type

```
cin >> data;
```

- Can be chained

```
cin >> data1 >> data2;
```

- Can fail if the entered data cannot be interpreted

data could have an undetermined value

Section Overview

Variables and Constants

- Declaring variables
- C++ primitive types
 - integer
 - floating point
 - boolean
 - character
- `sizeof` operator

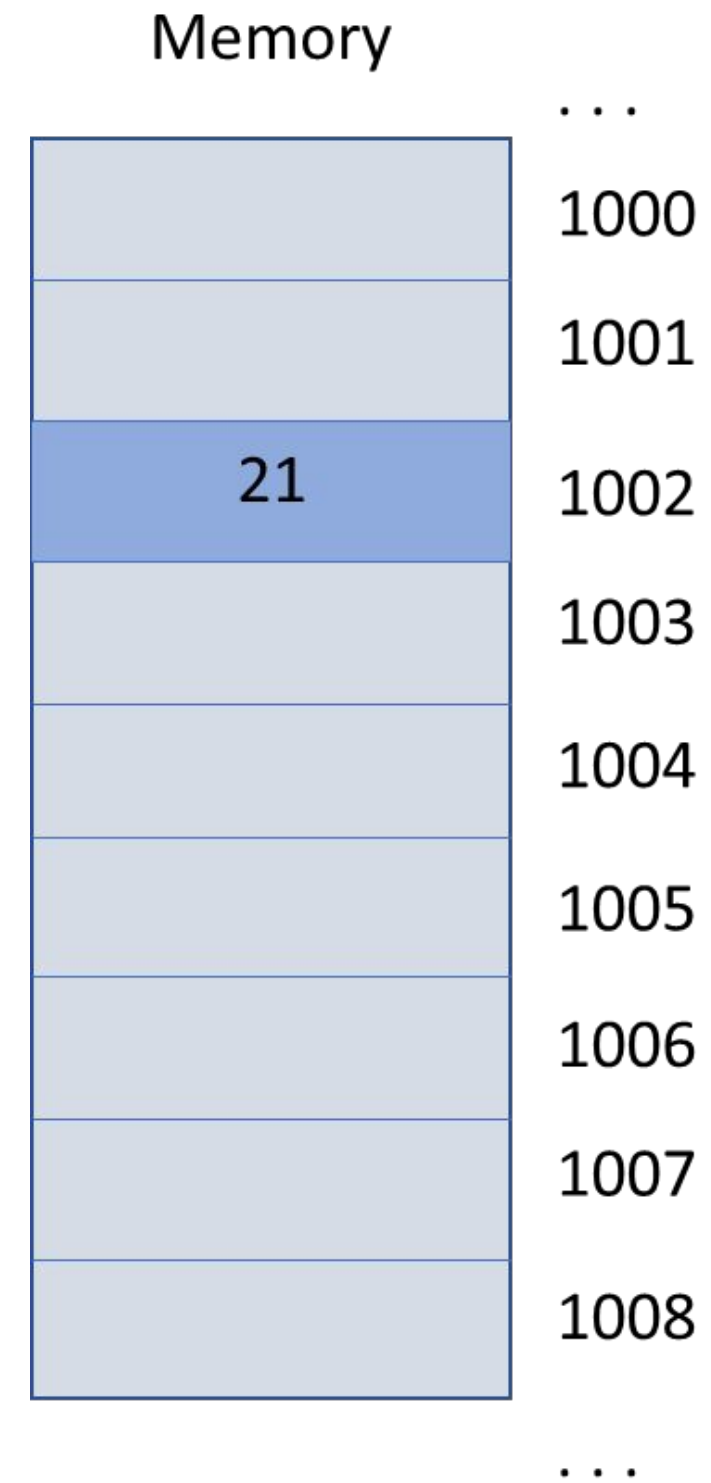
Section Overview

Variables and Constants

- What is a constant?
- Declaring constants
- Literal constants
- Constant expressions

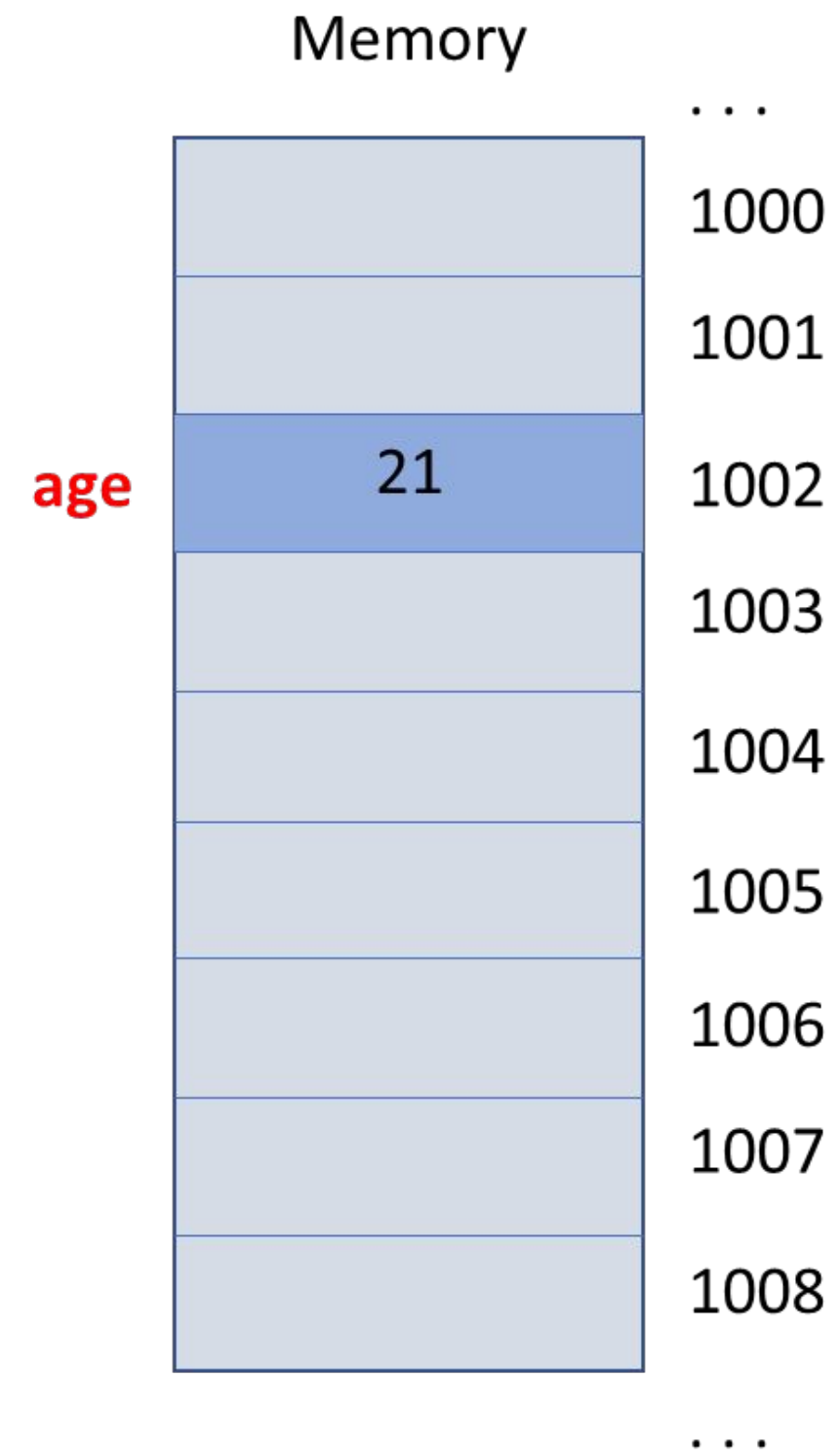
What is a variable?

move 21 to location 1002



What is a variable?

move 21 to age



What is a variable?

- A variable is an abstraction for a memory location
- Allow programmers to use meaningful names and not memory addresses
- Variables have
 - Type – their category (integer, real number, string, Person, Account...)
 - Value – the contents (10, 3.14, "Frank"...)
- Variables **must** be declared before they are used
- A variables value may change

```
age = 21;    // Compiler error
```

```
int age;
```

```
age = 21;
```

Declaring and Initializing Variables

Declaring Variables

```
VariableType VariableName;
```

```
int age;  
double rate;  
string name;  
  
Account franks_account;  
Person james;
```

Declaring and Initializing Variables

Naming Variables

- Can contain letters, numbers, and underscores
- Must begin with a letter or underscore (`_`)
 - cannot begin with a number
- Cannot use C++ reserved keywords
- Cannot redeclare a name in the same scope
 - remember that C++ is case sensitive

Declaring and Initializing Variables

Naming Variables

Legal	Illegal
Age	int
age	\$age
_age	2014_age
My_age	My age
your_age_in_2014	Age+1
INT	cout
Int	return

Declaring and Initializing Variables

Naming Variables – Style and Best Practices

- Be consistent with your naming conventions
 - myVariableName vs. my_variable_name
 - avoid beginning names with underscores
- Use meaningful names
 - not too long and not too short
- Never use variables before initializing them
- Declare variables close to when you need them in your code

Declaring and Initializing Variables

Initializing Variables

```
int age; // uninitialized
```

```
int age = 21; // C-like initialization
```

```
int age (21); // Constructor initialization
```

```
int age {21}; // C++11 list initialization syntax
```

C++ Primitive Data Types

- Fundamental data types implemented directly by the C++ language
- Character types
- Integer types
 - signed and unsigned
- Floating-point types
- Boolean type
- Size and precision is often compiler-dependent
 - `#include <climits>`

C++ Primitive Data Types

Type sizes

- Expressed in bits
- The more bits the more values that can be represented
- The more bits the more storage required

Size (in bits)	Representable Values	
8	256	2^8
16	65,536	2^{16}
32	4,294,967,296	2^{32}
64	18,446,744,073,709,551,615	2^{64}

C++ Primitive Data Types

Character Types

- Used to represent single characters, 'A', 'X', '@'
- Wider types are used to represent wide character sets

Type Name	Size / Precision
char	Exactly one byte. At least 8 bits.
char16_t	At least 16 bits.
char32_t	At least 32 bits.
wchar_t	Can represent the largest available character set.

C++ Primitive Data Types

Integer Types

- Used to represent whole numbers
- Signed and unsigned versions

C++ Primitive Data Types

Integer Types

Type Name	Size / Precision
<i>signed short int</i>	At least 16 bits.
<i>signed int</i>	At least 16 bits.
<i>signed long int</i>	At least 32 bits.
<i>signed long long int</i>	At least 64 bits

Type Name	Size / Precision
unsigned short int	At least 16 bits.
unsigned int	At least 16 bits.
unsigned long int	At least 32 bits.
unsigned long long int	At least 64 bits

C++ Primitive Data Types

Floating-point Type

- Used to represent non-integer numbers
- Represented by mantissa and exponent (scientific notation)
- Precision is the number of digits in the mantissa
- Precision and size are compiler dependent

Type Name	Size / Typical Precision	Typical Range
float	/ 7 decimal digits	1.2×10^{-38} to 3.4×10^{38}
double	No less than float / 15 decimal digits	2.2×10^{-308} to 1.8×10^{308}
long double	No less than double / 19 decimal digits	3.3×10^{-4932} to 1.2×10^{4932}

C++ Primitive Data Types

Boolean Type

- Used to represent true and false
- Zero is false.
- Non-zero is true.

Type Name	Size / Precision
bool	Usually 8 bits true or false (C++ keywords)

Using the `sizeof` operator

- The `sizeof` operator
 - determines the size in bytes of a type or variable
- Examples:

```
sizeof(int)
```

```
sizeof(double)
```

```
sizeof(some_variable)
```

```
sizeof some_variable
```

Using the `sizeof` operator

`<climits>` and `<cfloat>`

- The `climits` and `cfloat` include files contain size and precision information about your implementation of C++

```
INT_MAX  
INT_MIN  
LONG_MIN  
LONG_MAX  
FLT_MIN  
FLT_MAX  
... 
```


What is a constant?

- Like C++ variables
 - Have names
 - Occupy storage
 - Are usually typed

However, their value cannot change once declared!

Types of constants in C++

- Literal constants
- Declared constants
 - `const` keyword
- Constant expressions
 - `constexpr` keyword
- Enumerated constants
 - `enum` keyword
- Defined constants
 - `#define`

Types of constants in C++

Literal constants

- The most obvious kind of constant

`x = 12;`

`y = 1.56;`

`name = "Frank";`

`middle_initial = 'J';`

Types of constants in C++

Literal constants

- Integer Literal Constants

12 - an integer

12U - an unsigned integer

12L - a long integer

12LL - a long long integer

Types of constants in C++

Literal constants

- Floating-point Literal Constants

12.1 - a double

12.1F - a float

12.1L - a long double

Types of constants in C++

Literal constants

- Character Literal Constants (escape codes)

\n - newline

\r - return

\t - tab

\b - backspace

\' - single quote

\\" - double quote

\\ - backslash

```
cout << "Hello\tthere\nmy friend\n";
```

```
Hello    there
```

```
my friend
```

Types of constants in C++

Declared constants

- Constants declared using the `const` keyword

```
const double pi {3.1415926};
```

```
const int months_in_year {12};
```

```
pi = 2.5; // Compiler error
```

Types of constants in C++

Defined constants

- Constants declared using the `const` keyword

```
#define pi 3.1415926
```

Don't use defined constants in Modern C++

Section Overview

Arrays and Vectors

- Arrays
 - What they are
 - Why we use arrays
 - Declaration and initialization
 - Accessing array elements
- Multi-dimensional arrays
- Vectors
 - What they are
 - Advantages vs. arrays
 - Declaration and initialization

Arrays

What is an array?

- Compound data type or data structure
 - Collection of elements
- All elements are of the same type
- Each element can be accessed directly

Arrays

Why do we need arrays?

```
int test_score_1 {0};  
int test_score_2 {0};  
int test_score_3 {0};
```

Arrays

Why do we need arrays?

```
int test_score_1 {0};  
int test_score_2 {0};  
int test_score_3 {0};  
int test_score_4 {0};  
int test_score_5 {0};  
.  
.  
.  
int test_score_100 {0};
```

Arrays

Characteristics

- Fixed size
- Elements are all the same type
- Stored contiguously in memory
- Individual elements can be accessed by
 - their position or index
- First element is at index 0
- Last element is at index size-1
- No checking to see if you are out of bounds
- Always initialize arrays
- Very efficient
- Iteration (looping) is often used to process

test_scores	
100	[0]
95	[1]
87	[2]
80	[3]
100	[4]
83	[5]
89	[6]
92	[7]
100	[8]
95	[9]

Arrays

Declaring

```
Element_Type array_name [constant number of elements];
```

```
int test_scores [5];  
  
int high_score_per_level [10];  
  
const int days_in_year {365};  
double hi_temperatures [days_in_year];
```

Arrays

Initialization

```
Element_Type array_name [number of elements] {init list}
```

```
int test_scores [5] {100,95,99,87,88};  
  
int high_score_per_level [10] {3,5};           // init to 3,5 and remaining to 0  
  
const double days_in_year {365};  
double hi_temperatures [days_in_year] {0}; // init all to zero  
  
int another_array [] {1,2,3,4,5};           // size automatically calculated
```

Arrays

Accessing array elements

```
array_name [element_index]
```

```
test_scores [1]
```

```
int test_scores [5] {100,95,99,87,88};
```

```
cout << "First score at index 0: " << test_scores[0] << endl;
```

```
cout << "Second score at index 1: " << test_scores[1] << endl;
```

```
cout << "Third score at index 2: " << test_scores[2] << endl;
```

```
cout << "Fourth score at index 3: " << test_scores[3] << endl;
```

```
cout << "Fifth score at index 4: " << test_scores[4] << endl;
```


Arrays

Changing the contents of array elements

```
array_name [element_index]
```

```
int test_scores [5] {100,95,99,87,88};  
  
cin >> test_scores[0];  
cin >> test_scores[1];  
cin >> test_scores[2];  
cin >> test_scores[3];  
cin >> test_scores[4];  
  
test_scores[0] = 90;           // assignment statement
```

Arrays

How does it work?

- The name of the array represents the location of the first element in the array (index 0)
- The [index] represents the offset from the beginning of the array
- C++ simply performs a calculation to find the correct element
- Remember – no bounds checking!

Arrays

Declaring multi-dimensional arrays

```
Element_Type array_name [dim1_size][dim2_size]
```

```
int movie_rating [3][4];
```

Arrays

Multi-dimensional arrays

```
const int rows {3};  
const int cols {4};  
int movie_rating [rows][cols];
```

		<i>movie</i> (second index)			
		0	1	2	3
<i>reviewer</i> (first index)	0	0	4	3	5
	1	2	3	3	5
	2	1	4	4	5

Arrays

Accessing array elements in multi-dimensional arrays

```
cin >> movie_rating [1][2];  
cout << movie_rating [1][2];
```

		<i>movie</i> (second index)			
		0	1	2	3
<i>reviewer</i> (first index)	0	0	4	3	5
	1	2	3	5	5
	2	1	4	4	5

Arrays

Initializing multi-dimensional arrays

```
int movie_rating [3][4]
{
    { 0, 4, 3, 5},
    { 2, 3, 3, 5},
    { 1, 4, 4, 5}
};
```

	0	1	2	3
0	0	4	3	5
1	2	3	3	5
2	1	4	4	5

Vectors

- Suppose we want to store test scores for my school
- I have no way of knowing how many students will register next year
- Options:
 - Pick a size that you are not likely to exceed and use static arrays
 - Use a dynamic array such as vector

Vectors

What is a vector?

- Container in the C++ Standard Template Library
- An array that can grow and shrink in size at execution time
- Provides similar semantics and syntax as arrays
- Very efficient
- Can provide bounds checking
- Can use lots of cool functions like sort, reverse, find, and more.

Vectors

Declaring

```
#include <vector>
using namespace std;

vector <char> vowels;

vector <int> test_scores;
```

Vectors

Declaring

```
vector <char> vowels (5);
```

```
vector <int> test_scores (10);
```

Vectors

Initializing

```
vector <char> vowels {'a' , 'e' , 'i' , 'o' , 'u' };
```

```
vector <int> test_scores {100, 98, 89, 85, 93};
```

```
vector <double> hi_temperatures (365, 80.0);
```

Vectors

Characteristics

- Dynamic size
- Elements are all the same type
- Stored contiguously in memory
- Individual elements can be accessed by
 - their position or index
- First element is at index 0
- Last element is at index size-1
- [] - no checking to see if you are out of bounds
- Provides many useful function that do bounds check
- Elements initialized to zero
- Very efficient
- Iteration (looping) is often used to process

Vectors

Accessing vector elements – array syntax

```
vector_name [element_index]
```

```
test_scores [1]
```

```
vector <int> test_scores {100,95,99,87,88};
```

```
cout << "First score at index 0: " << test_scores[0] << endl;
```

```
cout << "Second score at index 1: " << test_scores[1] << endl;
```

```
cout << "Third score at index 2: " << test_scores[2] << endl;
```

```
cout << "Fourth score at index 3: " << test_scores[3] << endl;
```

```
cout << "Fifth score at index 4: " << test_scores[4] << endl;
```


Vectors

Accessing vector elements – vector syntax

```
vector_name.at(element_index)
```

```
test_scores.at(1)
```

```
vector <int> test_scores {100,95,99,87,88};
```

```
cout << "First score at index 0: " << test_scores.at(0) << endl;
```

```
cout << "Second score at index 1: " << test_scores.at(1) << endl;
```

```
cout << "Third score at index 2: " << test_scores.at(2) << endl;
```

```
cout << "Fourth score at index 3: " << test_scores.at(3) << endl;
```

```
cout << "Fifth score at index 4: " << test_scores.at(4) << endl;
```

Vectors

Changing the contents of vector elements – vector syntax

```
vector_name.at(element_index)
```

```
vector<int> test_scores {100,95,99,87,88};  
  
cin >> test_scores.at(0);  
cin >> test_scores.at(1);  
cin >> test_scores.at(2);  
cin >> test_scores.at(3);  
cin >> test_scores.at(4);  
  
test_scores.at(0) = 90;           // assignment statement
```

Vectors

So, when do they grow as needed?

```
vector_name.push_back(element)
```

```
vector <int> test_scores {100,95,99};    // size is 3

test_scores.push_back(80);                // 100, 95, 99, 80
test_scores.push_back(90);                // 100, 95, 99, 80, 90
```

Vector will automatically allocate the required space!

Vectors

What if you are out of bounds?

- Arrays never do bounds checking
- Many vector methods provide bounds checking
- An exception and error message is generated

```
vector<int> test_scores { 100,95 };
```

```
cin >> test_scores.at(5);
```

```
terminate called after throwing an instance of 'std::out_of_range'
```

```
what():  vector::_M_range_check: __n (which is 5) >= this->size() (which is 2)
```

```
This application has requested the Runtime to terminate it in an unusual way.
```

```
Please contact the application's support team for more information.
```

Section Overview

Expressions, Statements and Operators

- Expressions
- Statements and block statements
- Operators
 - Assignment
 - Arithmetic
 - Increment and decrement
 - Equality
 - Relational
 - Logical
 - Compound assignment
 - Precedence

Expressions and Statements

Expressions

An expression is:

- The most basic building block of a program
- “a sequence of operators and operands that specifies a computation”
- Computes a value from a number of operands
- There is much, much more to expressions – not necessary at this level

Expressions and Statements

Expressions - examples

34 // literal

favorite_number // variable

1.5 + 2.8 // addition

2 * 5 // multiplication

a > b // relational

a = b // assignment

Expressions and Statements

Statements

A statement is:

- A complete line of code that performs some action
- Usually terminated with a semi-colon
- Usually contain expressions
- C++ has many types of statements
 - expression, null, compound, selection, iteration,
 - declaration, jump, try blocks, and others.

Expressions and Statements

Statements - examples

```
int x;                // declaration

favorite_number = 12; // assignment

1.5 + 2.8;           // expression

x = 2 * 5;           // assignment

if (a > b) cout << "a is greater than b"; // if

;
```

Using Operators

- C++ has a rich set of operators
 - unary, binary, ternary
- Common operators can be grouped as follows:
 - assignment
 - arithmetic
 - increment/decrement
 - relational
 - logical
 - member access
 - other

Assignment Operator (=)

`lhs = rhs`

- `rhs` is an expression that is evaluated to a value
- The value of the `rhs` is stored to the `lhs`
- The value of the `rhs` must be type compatible with the `lhs`
- The `lhs` must be assignable
- Assignment expression is evaluated to what was just assigned
- More than one variable can be assigned in a single statement

Mixed Type Expressions

- C++ operations occur on same type operands
- If operands are of different types, C++ will convert one
- Important! since it could affect calculation results
- C++ will attempt to automatically convert types (coercion).

If it can't, a compiler error will occur

Mixed Type Expressions

Conversions

- Higher vs. Lower types are based on the size of the values the type can hold
 - long double, double, float, unsigned long, long, unsigned int, int
 - short and char types are always converted to int
- Type Coercion: conversion of one operand to another data type
- Promotion: conversion to a higher type
 - Used in mathematical expressions
- Demotion: conversion to a lower type
 - Used with assignment to lower type

Mixed Type Expressions

Examples

- `lower op higher` **the lower is promoted to a higher**

`2 * 5.2`

`2` is promoted to `2.0`

- `lower = higher;` **the higher is demoted to a lower**

`int num {0};`

`num = 100.2;`

Mixed Type Expressions

Explicit Type Casting – `static_cast<type>`

```
int total_amount {100};  
int total_number {8};  
double average {0.0};  
  
average = total_amount / total_number;  
cout << average << endl;           // displays 12  
  
average = static_cast<double>(total_amount) / total_number;  
cout << average << endl;           // displays 12.5
```

Testing for Equality

The `==` and `!=` operators

- Compares the values of 2 expressions
- Evaluates to a Boolean (True or False, 1 or 0)
- Commonly used in control flow statements

```
expr1 == expr2
```

```
expr1 != expr2
```

```
100 == 200
```

```
num1 != num2
```


Testing for Equality

The `==` and `!=` operators

```
bool result {false};

result = (100 == 50+50);

result = (num1 != num2);

cout << (num1 == num2) << endl;    // 0 or 1
cout << std::boolalpha;
cout << (num1 == num2) << endl;    // true or false
cout << std::noboolalpha;
```

Relational Operators

`expr1 op expr2`

Operator	Meaning
<code>></code>	greater than
<code>>=</code>	greater than or equal to
<code><</code>	less than
<code><=</code>	less than or equal to
<code><=></code>	three-way comparison (C++20)

Logical Operators

Operator	Meaning
<code>not</code> <code>!</code>	negation
<code>and</code> <code>&&</code>	logical and
<code>or</code> <code> </code>	logical or

Logical Operators

not (!)

expression a	not a !a
true	false
false	true

Logical Operators

and (&&)

expression a	expression b	a and b a && b
true	true	true
true	false	false
false	true	false
false	false	false

Logical Operators

or (||)

expression a	expression b	a or b a b
true	true	true
true	false	true
false	true	true
false	false	false

Logical Operators

Precedence

- `not` has higher precedence than `and`
- `and` has higher precedence than `or`
- `not` is a unary operator
- `and` and `or` are binary operators

Logical Operators

Examples

```
num1 >= 10 && num1 < 20  
num1 <= 10 || num1 >= 20
```

```
!is_raining && temperature > 32.0
```

```
is_raining || is_snowing
```

```
temperature > 100 && is_humid || is_raining
```


Logical Operators

Short-Circuit Evaluation

- When evaluating a logical expression C++ stops as soon as the result is known

`expr1 && expr2 && expr3`

`expr1 || expr2 || expr3`

Compound Assignment

op=

Operator	Example	Meaning
<code>+=</code>	<code>lhs += rhs;</code>	<code>lhs = lhs + (rhs);</code>
<code>-=</code>	<code>lhs -= rhs;</code>	<code>lhs = lhs - (rhs);</code>
<code>*=</code>	<code>lhs *= rhs;</code>	<code>lhs = lhs * (rhs);</code>
<code>/=</code>	<code>lhs /= rhs;</code>	<code>lhs = lhs / (rhs);</code>
<code>%=</code>	<code>lhs %= rhs;</code>	<code>lhs = lhs % (rhs);</code>
<code>>>=</code>	<code>lhs >>= rhs;</code>	<code>lhs = lhs >> (rhs);</code>
<code><<=</code>	<code>lhs <<= rhs;</code>	<code>lhs = lhs << (rhs);</code>
<code>&=</code>	<code>lhs &= rhs;</code>	<code>lhs = lhs & (rhs);</code>
<code>^=</code>	<code>lhs ^= rhs;</code>	<code>lhs = lhs ^ (rhs);</code>
<code> =</code>	<code>lhs = rhs;</code>	<code>lhs = lhs (rhs);</code>

Logical Operators

Examples

```
lhs op= rhs; // lhs = lhs op (rhs);
```

```
a += 1;      // a = a + 1;
```

```
a /= 5;      // a = a / 5;
```

```
a *= b + c;  // a = a * (b + c);
```

```
cost += items * tax;
```

```
//cost = cost + (items * tax);
```


Operator Precedence (not a complete list)

Higher to lower

Operator	Associativity
<code>[] -> . ()</code>	left to right
<code>++ -- not -(unary) *(de-ref) & sizeof</code>	right to left
<code>* / %</code>	left to right
<code>+ -</code>	left to right
<code><< >></code>	left to right
<code>< <= > >=</code>	left to right
<code>== !=</code>	left to right
<code>&</code>	left to right
<code>^</code>	left to right
<code> </code>	left to right
<code>&&</code>	left to right
<code> </code>	left to right
<code>= op= ?:</code>	right to left

Operator Precedence

What is associativity?

- Use precedence rules when adjacent operators are different

`expr1 op1 expr2 op2 expr3 // precedence`

- Use associativity rules when adjacent operators have the same precedence

`expr1 op1 expr2 op1 expr3 // associativity`

- Use parenthesis to absolutely remove any doubt

Operator Precedence

Example

```
result = num1 + num2 * num3;
```

```
result = ( num1 + (num2 * num3) );
```

```
result1 = num1 + num2 - num3;
```

```
result1 = ( (num1 + num2) - num3 );
```

Section Overview

Controlling Program Flow

- Sequence
 - Ordering statements sequentially
- Selection
 - Making decisions
- Iteration
 - Looping or repeating

Selection – Decision Making

- `if` statement
- `if-else` statement
- Nested `if` statements
- `switch` statement
- Conditional operator `?:`

Iteration - Looping

- `for` loop
- Range-based `for` loop
- `while` loop
- `do-while` loop
- `continue` and `break`
- Infinite loops
- Nested loops

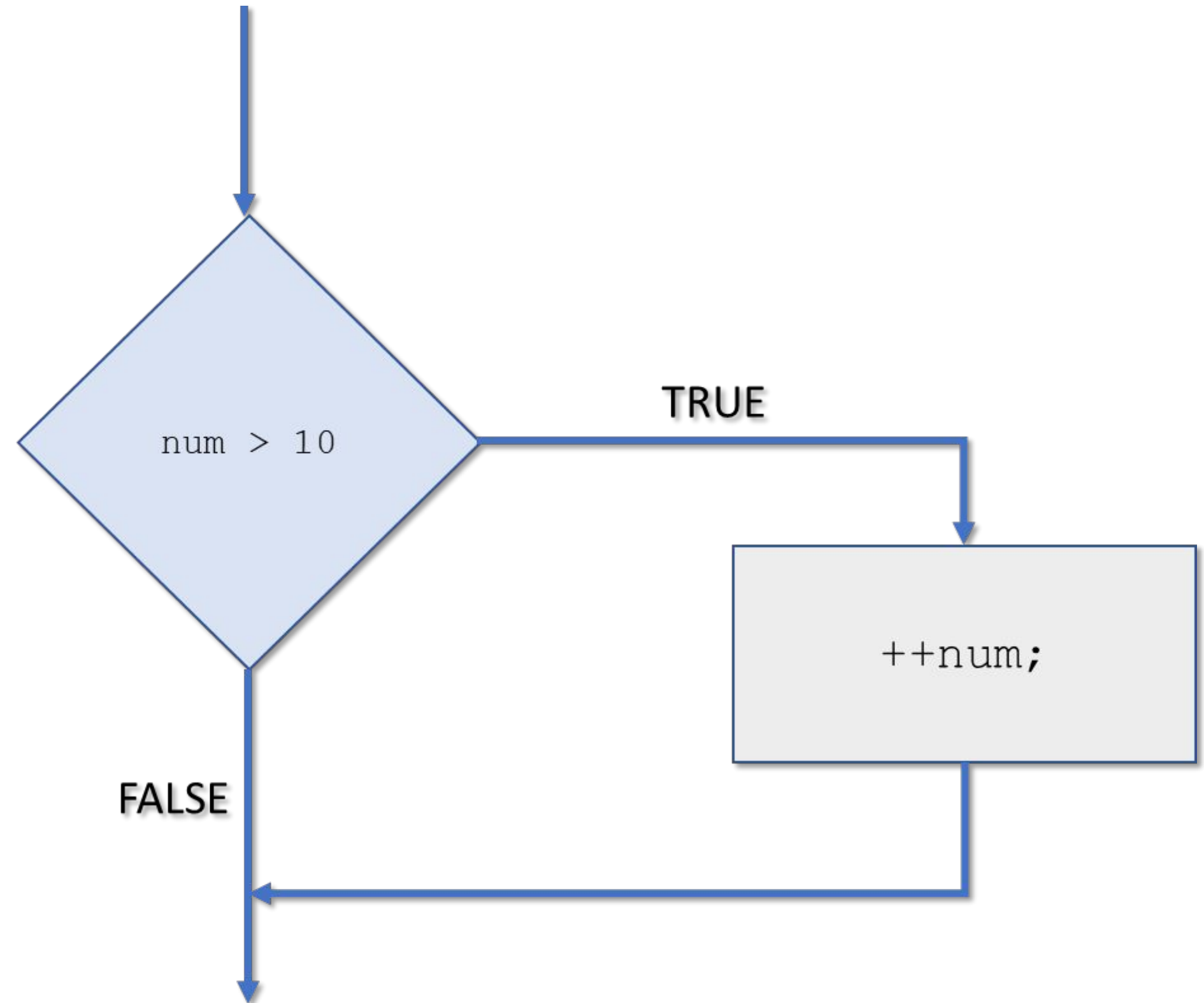
if statement

```
if (expr)
    statement;
```

- If the expression is true then execute the statement
- If the expression is false then skip the statement

if statement

```
if (num > 10)
    ++num;
```



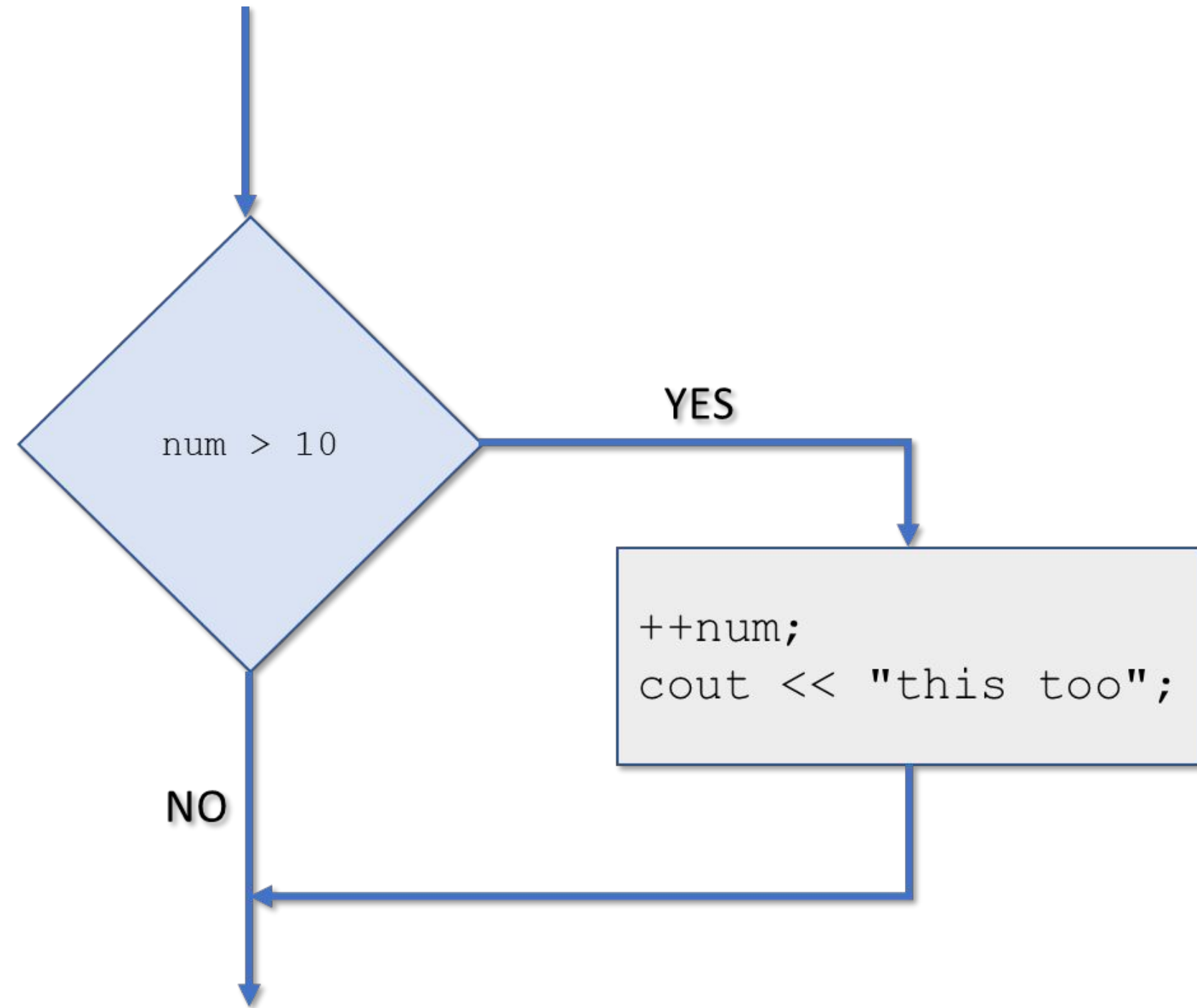
if statement

```
if (selection == 'A')  
    cout << "You selected A";  
  
if (num > 10)  
    cout << "num is greater than 10";  
  
if (health < 100 && player_healed)  
    health = 100;
```

if statement

block statement

```
if (num > 10) {  
    ++num;  
    cout << "this too";  
}
```



Block statement

```
{  
    //variable declarations  
    statement1;  
    statement2;  
    . . .  
}
```

- Create a block of code by including more than one statement in code block { }
- Blocks can also contain variable declarations
- These variables are visible only within the block – local scope

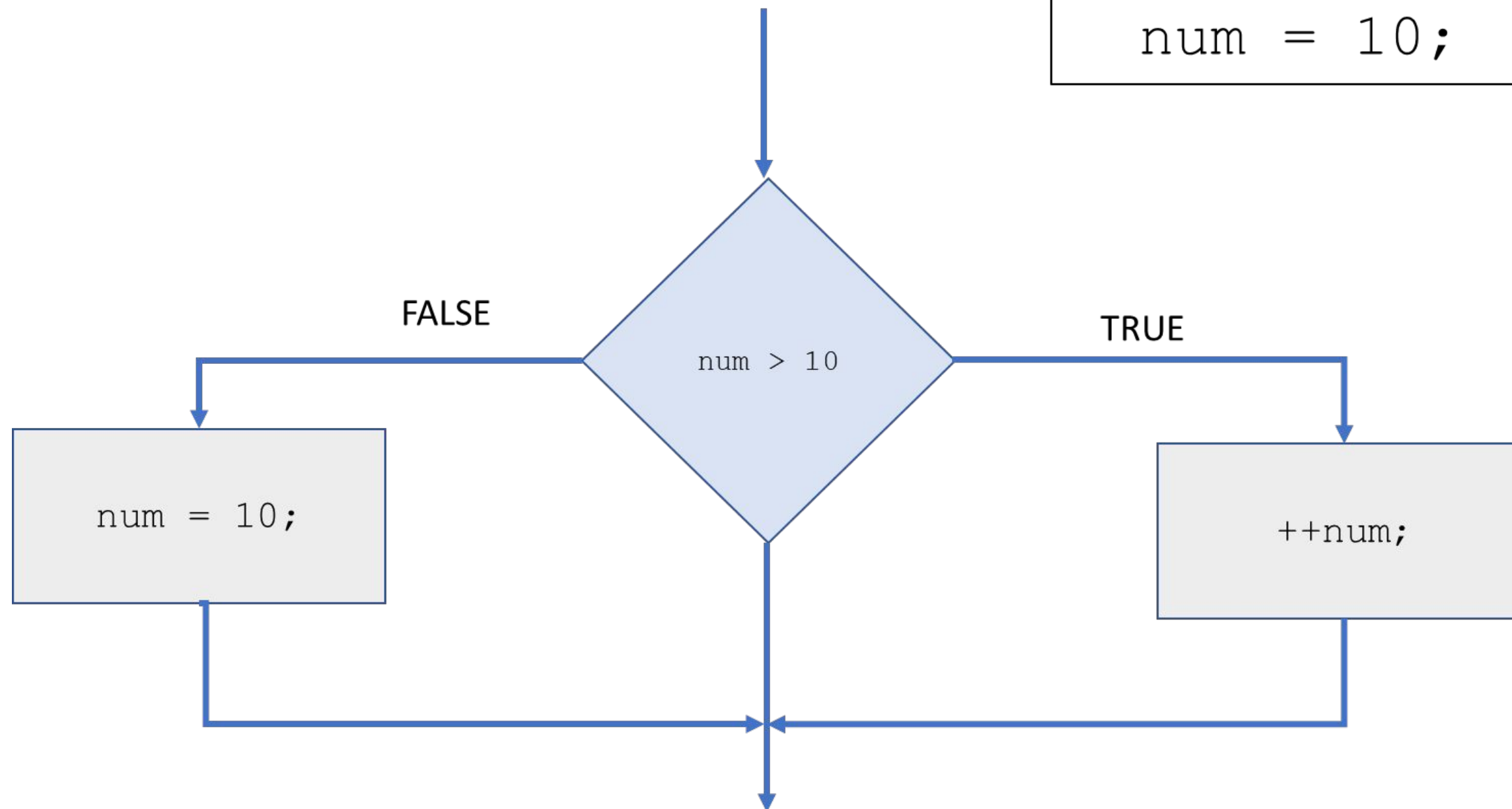
if-else statement

```
if (expr)
    statement1;
else
    statement2;
```

- If the expression is **true** then execute **statement1**
- If the expression is **false** then execute **statement2**

if-else statement

```
if (num > 10)
    ++num;
else
    num = 10;
```



if-else statement

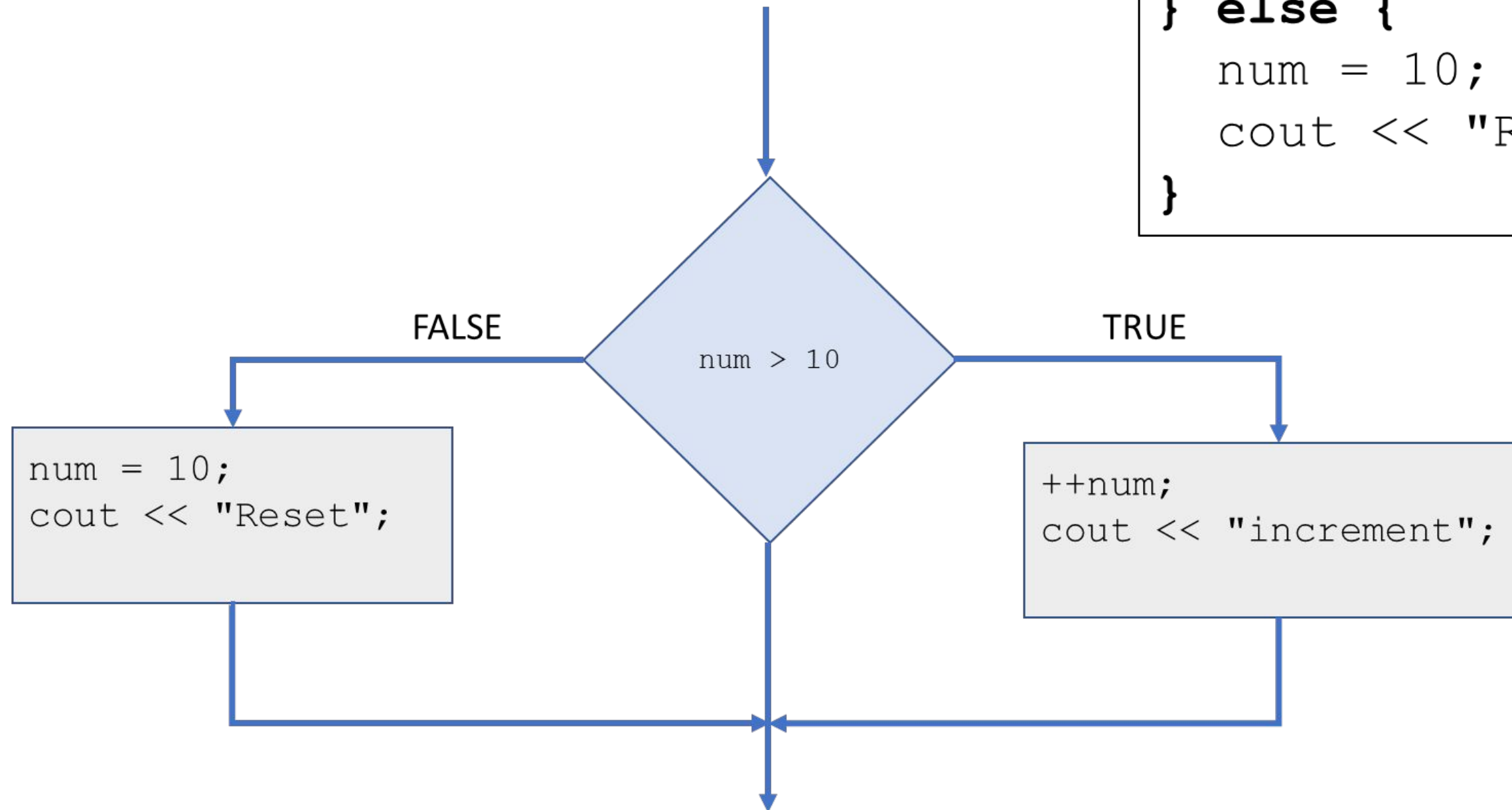
```
if (num > 10)
    cout << "num is greater than 10";
else
    cout << "num is NOT greater than 10";

if (health < 100 && heal_player)
    health = 100;
else
    ++health;
```

if-else statement

block statement

```
if (num > 10) {  
    ++num;  
    cout << "increment";  
} else {  
    num = 10;  
    cout << "Reset";  
}
```



if-else-if construct

block statement

```
if (score > 90)
    cout << "A";
else if (score > 80)
    cout << "B";
else if (score > 70)
    cout << "C";
else if (score > 60)
    cout << "D";
else                                // all others must be F
    cout << "F";
cout << "Done";
```


Nested `if` statement

```
if (expr1)
    if (expr2)
        statement1;
    else
        statement2;
```

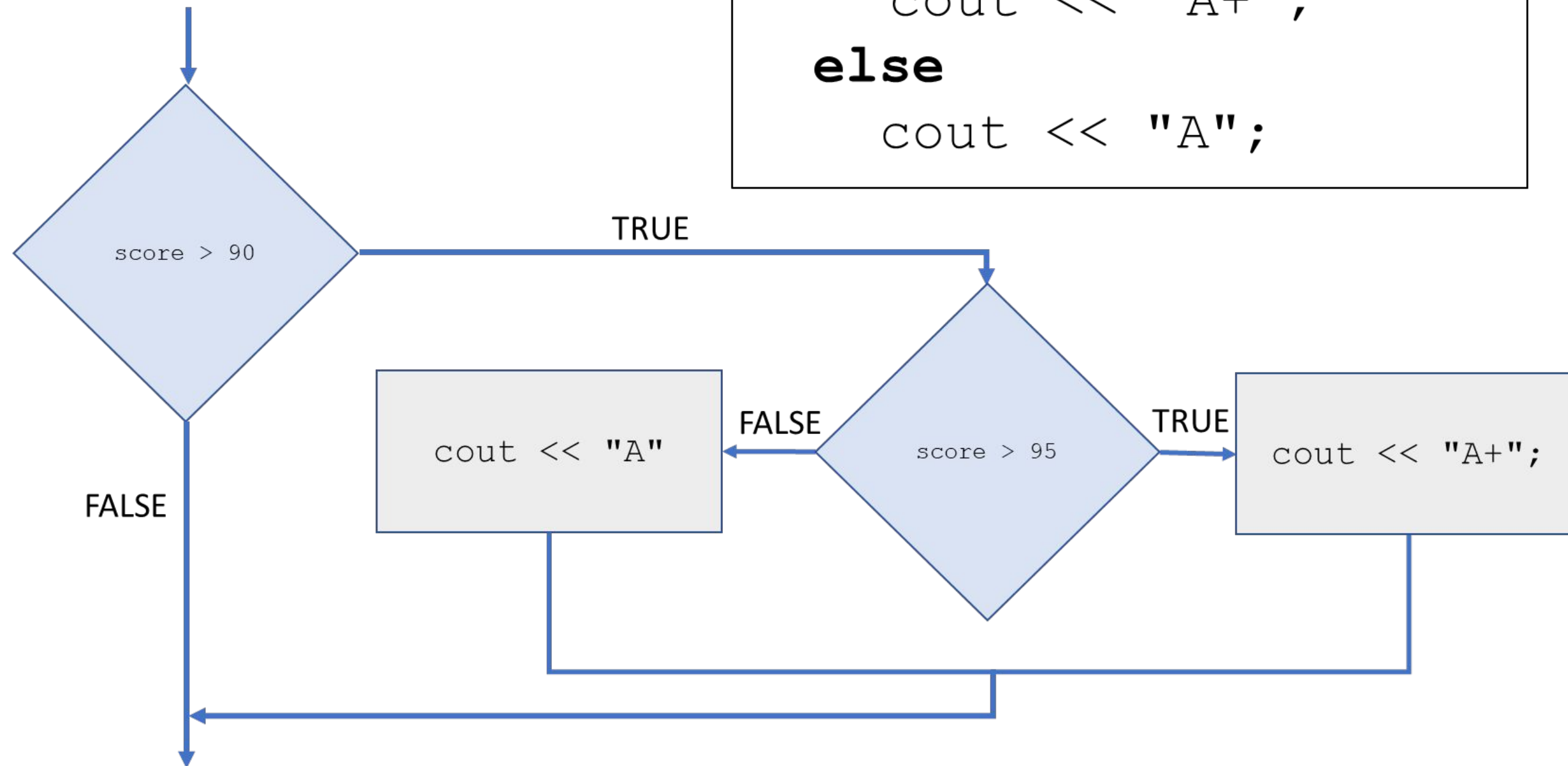
- **`if`** statement is nested within another
- Allows testing of multiple conditions
- **`else`** belongs to the closest **`if`**

Nested `if` statement

```
if (score > 90)
    if (score > 95)
        cout << "A+";
    else
        cout << "A";
else
    cout << "Sorry, No A";
```

Nested if statement

```
if (score > 90)
    if (score > 95)
        cout << "A+";
    else
        cout << "A";
```



Nested if statement

```
if (score_frank != score_bill) {  
    if (score_frank > score_bill) {  
        cout << "Frank Wins" << endl;  
    } else {  
        cout << "Bill Wins" << endl;  
    }  
} else {  
    cout << "Looks like a tie!" << endl;  
}
```

The `switch` statement

```
switch (integer_control_expr) {  
    case expression_1: statement_1; break;  
    case expression_2: statement_2; break;  
    case expression_3: statement_3; break;  
    . . .  
    case expression_n: statement_n; break;  
    default: statement_default;  
}
```


The `switch` statement

example

```
switch (selection) {  
    case '1': cout << "1 selected";  
                break;  
    case '2': cout << "2 selected";  
                break;  
    case '3':  
    case '4': cout << "3 or 4 selected";  
                break;  
    default:    cout << "1,2,3,4 NOT selected";  
}
```

The `switch` statement

fall-through example

```
switch (selection) {  
    case '1': cout << "1 selected";  
  
    case '2': cout << "2 selected";  
  
    case '3': cout << "3 selected";  
  
    case '4': cout << "4 selected";  
            break;  
    default:   cout << "1,2,3,4 NOT selected";  
}
```

The `switch` statement

with an enumeration

```
enum Color {  
    red, green, blue  
};  
Color screen_color {green};
```

```
switch (screen_color) {  
    case red:    cout << "red"; break;  
    case green: cout << "green"; break;  
    case blue:  cout << "blue"; break;  
    default:   cout << "should never execute";  
}
```


The `switch` statement

- The control expression must evaluate to an integer type
- The case expressions must be constant expressions that evaluate to integer or integers literals
- Once a match occurs all following case sections are executed UNTIL a `break` is reached the switch complete
- Best practice – provide `break` statement for each case
- Best practice – `default` is optional, but should be handled

Conditional Operator

?:

`(cond_expr) ? expr1 : expr2`

- `cond_expr` evaluates to a boolean expression
 - If `cond_expr` is true then the value of `expr1` is returned
 - If `cond_expr` is false then the value of `expr2` is returned
- Similar to if-else construct
- Ternary operator
- Very useful when used inline
- Very easy to abuse!

Conditional Operator

example

```
int a{10}, b{20};  
int score{92};  
int result {};  
  
result = (a > b) ? a : b;  
  
result = (a < b) ? (b-a) : (a-b);  
  
result = (b != 0) ? (a/b) : 0;  
  
cout << ((score > 90) ? "Excellent" : "Good ");
```

Looping

iteration

- The third basic building block of programming
 - sequence, selection, **iteration**
- Iteration or repetition
- Allows the execution of a statement or block of statements repeatedly
- Loops are made up a loop condition and the body which contains the statements to repeat

Looping

Some typical use-cases

Execute a loop:

- a specific number of times
- for each element in a collection
- while a specific condition remains true
- until a specific condition becomes false
- until we reach the end of some input stream
- forever
- many, many more

C++ Looping Constructs

- `for` loop
 - iterate a specific number of times
- Range-based `for` loop
 - one iteration for each element in a range or collection
- `while` loop
 - iterate while a condition remains true
 - stop when the condition becomes false
 - check the condition at the beginning of every iteration
- `do-while` loop
 - iterate while a condition remains true
 - stop when the condition becomes false
 - check the condition at the end of every iteration

for Loop

```
for (initialization ; condition ; increment)
    statement;
```

```
for (initialization ; condition ; increment) {
    statement(s);
}
```

for Loop

```
int i {0};
```

```
for (i = 1 ; i <= 5 ; ++i)  
    cout << i << endl;
```

1

2

3

4

5

for Loop

```
for (int i {1} ; i <= 5 ; ++i)  
    cout << i << endl;
```

```
for (int i = 1 ; i <= 5 ; ++i)  
    cout << i << endl;
```

i = 100; // ERROR i only visible in the loop

for Loop

display even numbers

```
for (int i {1} ; i <= 10 ; ++i) {  
    if (i % 2 == 0)  
        cout << i << endl;
```

```
2  
4  
6  
8  
10
```

for Loop

array example

```
int scores [] {100,90,87};

for (int i {0} ; i < 3 ; ++i) {
    cout << scores[i] << endl;
}

for (int i {0} ; i <= 2 ; ++i) {
    cout << scores[i] << endl;
}
100
90
87
```

for Loop

comma operator

```
for (int i {1}, j {5} ; i <= 5 ; ++i, ++j) {  
    cout << i << " * " << j << " : " << (i * j) << endl;  
}
```

```
1 * 5 : 5  
2 * 6 : 12  
3 * 7 : 21  
4 * 8 : 32  
5 * 9 : 45
```

for Loop

some other details...

- The basic for loop is very clear and concise
- Since the for loop's expressions are all optional, it is possible to have
 - no initialization
 - no test
 - no increment

```
for (;;)
    cout << "Endless loop" << endl;
```


Range-based for Loop

Introduced in C++11

```
for (var_type var_name: sequence)
    statement; // can use var_name
```

```
for (var_type var_name: sequence) {
    statements; // can use var_name
}
```

Range-based for Loop

```
int scores [] {100, 90, 97};
```

```
for (int score : scores)  
    cout << score << endl;
```

```
100
```

```
90
```

```
97
```

Range-based for Loop

auto

```
int scores [] {100, 90, 97};
```

```
for (auto score : scores)  
    cout << score << endl;
```

100

90

97

Range-based for Loop

vector

```
vector<double> temps {87.2, 77.1, 80.0, 72.5};  
  
double average_temp {};  
double running_sum {};  
  
for (auto temp: temps)  
    running_sum += temp;  
  
average_temp = running_sum / temps.size();
```

Range-based for Loop

initializer list

```
double average_temp {};  
double running_sum {};  
int size {0};  
  
for (auto temp: {60.2, 80.1, 90.0, 78.2} ) {  
    running_sum += temp;  
    ++size;  
}  
average_temp = running_sum / size;
```

Range-based `for` Loop

string

```
for (auto c: "Frank")  
    cout << c << endl;
```

F
r
a
n
k

while Loop

```
while (expression)
    statement;
```

```
while (expression) {
    statement(s);
}
```

while Loop

```
int i {1};

while (i <= 5) {
    cout << i << endl;
    ++i; // important!
}

1
2
3
4
5
```


while Loop

even numbers

```
int i {1};

while (i <= 10) {
    if (i % 2 == 0)
        cout << i << endl;
    ++i;
}

2
4
6
8
10
```

while Loop

array example

```
int scores [] {100,90,87};  
int i {0};  
  
while (i < 3) {  
    cout << scores[i] << endl;  
    ++i;  
}  
  
100  
90  
87
```

while Loop

input validation

```
int number {};  
  
cout << "Enter an integer less than 100: ";  
cin >> number;  
  
while (number >= 100) {    // !(number < 100)  
    cout << "Enter an integer less than 100";  
    cin >> number;  
}  
  
cout << "Thanks" << endl;
```


while Loop

input validation

```
int number {};  
  
cout << "Enter an integer between 1 and 5: ";  
cin >> number;  
  
while (number <= 1 || number >= 5) {  
    cout << "Enter an integer between 1 and 5: ";  
    cin >> number;  
}  
  
cout << "Thanks" << endl;
```

while Loop

input validation – boolean flag

```
bool done {false};
int number {0};

while (!done) {
    cout << "Enter an integer between 1 and 5: ";
    cin >> number;
    if (number <=1 || number >=5)
        cout << "Out of range, try again" << endl;
    else {
        cout << "Thanks!" << endl;
        done = true;
    }
}
```

do-while Loop

```
do {  
    statements;  
} while (expression);
```

do-while Loop

input validation

```
int number {};  
do {  
  
    cout << "Enter an integer between 1 and 5: ";  
    cin >> number;  
  
} while (number <= 1 || number >= 5) ;  
  
cout << "Thanks" << endl;
```

do-while Loop

area calculation with calculate another

```
char selection {};  
  
do {  
    double width {}, height {};  
    cout << "Enter width and height separated by a space :";  
    cin >> width >> height;  
  
    double area {width * height};  
    cout << "The area is " << area << endl;  
  
    cout << "Calculate another? (Y/N) : ";  
    cin >> selection;  
} while (selection == 'Y' || selection == 'y');  
cout << "Thanks!" << endl;
```


continue and break statements

- `continue`
 - no further statements in the body of the loop are executed
 - control immediately goes directly to the beginning of the loop for the next iteration
- `break`
 - no further statements in the body of the loop are executed
 - loop is immediately terminated
 - Control immediately goes to the statement following the loop construct

continue and break statements

```
vector<int> values {1,2,-1,3,-1,-99,7,8,10};  
for (auto val: values) {  
    if (val == -99)  
        break;  
    else if (val == -1)  
        continue;  
    else  
        cout << val << endl;  
}  
1  
2  
3
```

Infinite Loops

- Loops whose condition expression always evaluate to true
- Usually this is unintended and a programmer error
- Sometimes programmers use infinite loops and include and break statements in the body to control them
- Sometimes infinite loops are exactly what we need
 - Event loop in an event-driven program
 - Operating system

Infinite for Loops

```
for (;;)
    cout << "This will print forever" << endl;
```

Infinite while Loops

```
while (true)
    cout << "This will print forever" << endl;
```

Infinite do-while Loops

```
do {  
    cout << "This will print forever" << endl;  
} while (true);
```

Infinite while Loops

example

```
while (true) {  
    char again {};  
    cout << "Do you want to loop again? (Y/N) : ";  
    cin >> again;  
  
    if (again == 'N' || again == 'n')  
        break;  
}
```

Nested Loops

- Loop nested within another loop
- Can be many as many levels deep as the program needs
- Very useful with multi-dimensional data structures
- Outer loop vs. Inner loop

Nested Loops

```
for (outer_val {1}; outer_val <= 2 ; ++outer_val)
    for (inner_val {1}; inner_val <= 3; ++inner_val)
        cout << outer_val << ", " << inner_val << endl;
```

```
1, 1
1, 2
1, 3
2, 1
2, 2
2, 3
```

outer_val, inner_val

Note: inner loop loops "faster"

Nested Loops

Multiplication Table

```
for (int num1 {1}; num1 <=10 ; ++num1) {    // outer
    for (int num2 {1}; num2 <=10; ++num2) { // inner
        cout << num1 << " * " << num2
            << " = " << num1 * num2 << endl;
    }
    cout << "-----" << endl;
}
```

Displays 10 x 10 Multiplication Table

Nested Loops

2D Arrays – set all elements to 1000

```
int grid[5][3] {};  
  
for (int row {0}; row < 5; ++row ) {  
    for (int col {0}; col < 3; ++col ) {  
        grid[row][col] = 1000;  
    }  
}
```


Nested Loops

2D Arrays – display elements

```
for (int row {0}; row < 5; ++row ) {  
    for (int col {0}; col < 3; ++col ) {  
        cout << grid[row][col] << " ";  
    }  
    cout << endl;  
}
```

Nested Loops

2D Vector – display elements

```
vector<vector<int>> vector_2d
{
    {1, 2, 3},
    {10, 20, 30, 40},
    {100, 200, 300, 400, 500}
};

for (auto vec: vector_2d) {
    for (auto val: vec) {
        cout << val << " ";
    }
    cout << endl;
}
```

Output

```
1 2 3
10 20 30 40
100 200 300 400 500
```

Section Overview

Characters and Strings

- Character functions
- C-style Strings
- Working with C-style Strings
- C++ Strings
- Working with C++ Strings

Character Functions

```
#include <cctype>
```

```
#include <cctype>
```

```
function_name(char)
```

- Functions for testing characters
- Functions for converting character case

Character Functions

Testing characters

<code>isalpha(c)</code>	True if <code>c</code> is a letter
<code>isalnum(c)</code>	True if <code>c</code> is a letter or digit
<code>isdigit(c)</code>	True if <code>c</code> is a digit
<code>islower(c)</code>	True if <code>c</code> is lowercase letter
<code>isprint(c)</code>	True if <code>c</code> is a printable character
<code>ispunct(c)</code>	True if <code>c</code> is a punctuation character
<code>isupper(c)</code>	True if <code>c</code> is an uppercase letter
<code>isspace(c)</code>	True if <code>c</code> is whitespace

Character Functions

Converting characters

<code>tolower(c)</code>	returns lowercase of c
<code>toupper(c)</code>	returns uppercase of c

C-style Strings

- Sequence of characters
 - contiguous in memory
 - implemented as an array of characters
 - terminated by a null character (null)
 - null – character with a value of zero
 - Referred to as zero or null terminated strings
- String literal
 - sequence of characters in double quotes, e.g. "Frank"
 - constant
 - terminated with null character

C-style Strings

"C++ is fun"

C	+	+		i	s		f	u	n	\0
---	---	---	--	---	---	--	---	---	---	----

C-style Strings

declaring variables

```
char my_name[] {"Frank"};
```

F	r	a	n	k	\0
---	---	---	---	---	----

```
my_name[5] = 'y'; // Problem
```

C-style Strings

declaring variables

```
char my_name[8] {"Frank"};
```

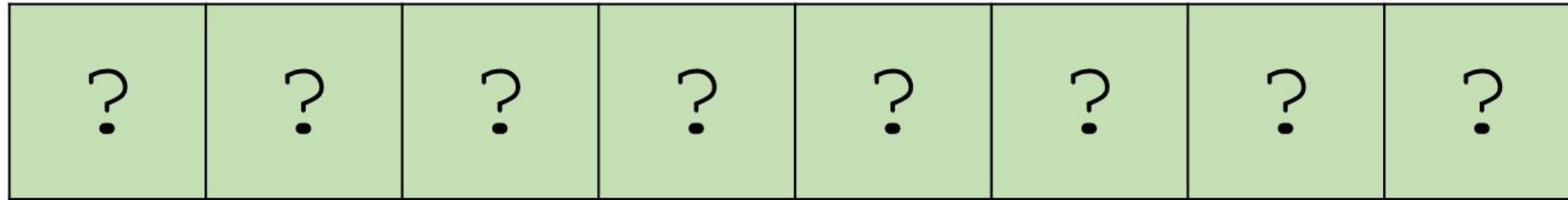
F	r	a	n	k	\0	\0	\0
---	---	---	---	---	----	----	----

```
my_name[5] = 'y'; // OK
```

C-style Strings

declaring variables

```
char my_name[8];
```



```
my_name = "Frank";           // Error
```

```
strcpy(my_name, "Frank");    // OK
```

#include <cstring>

Functions that work with C-style Strings

- Copying
- Concatenation
- Comparison
- Searching
- and others

#include <cstring>

A few examples

```
char str[80];  
  
strcpy(str, "Hello "); // copy  
  
strcat(str, "there"); // concatenate  
  
cout << strlen(str); // 11  
  
strcmp(str, "Another"); // > 0
```


#include <cstdlib>

General purpose functions

- Includes functions to convert C-style Strings to
 - integer
 - float
 - long
 - etc.

C++ Strings

- `std::string` is a Class in the Standard Template Library
 - `#include <string>`
 - `std` namespace
 - contiguous in memory
 - dynamic size
 - work with input and output streams
 - lots of useful member functions
 - our familiar operators can be used (`+`, `=`, `<`, `<=`, `>`, `>=`, `+=`, `==`, `!=`, `[]`...)
 - can be easily converted to C-style Strings if needed
 - safer

C++ Strings

Declaring and initializing

```
#include <string>
using namespace std;

string s1;                // Empty
string s2 {"Frank"};      // Frank
string s3 {s2};           // Frank
string s4 {"Frank", 3};    // Fra
string s5 {s3, 0, 2};     // Fr
string s6 (3, 'X');       // XXX
```


C++ Strings

Assignment =

```
string s1;  
s1 = "C++ Rocks!";  
  
string s2 {"Hello"};  
s2 = s1;
```

C++ Strings

Concatenation

```
string part1 {"C++"};  
string part2 {"is a powerful"};  
  
string sentence;  
  
sentence = part1 + " " + part2 + " language";  
           // C++ is a powerful language  
  
sentence = "C++" + " is powerful"; // Illegal
```

C++ Strings

Accessing characters [] and at() method

```
string s1;  
string s2 {"Frank"};  
  
cout << s2[0] << endl;    // F  
cout << s2.at(0) << endl; // F  
  
s2[0] = 'f';    // frank  
s2.at(0) = 'p'; // prank
```

C++ Strings

Accessing characters [] and at() method

```
string s1 {"Frank"};
```

```
for (char c: s1)  
    cout << c << endl;
```

F
r
a
n
k

C++ Strings

Accessing characters [] and at() method

```
string s1 {"Frank"};

for (int c: s1)
    cout << c << endl;

70    // F
114   // r
97    // a
110   // n
107   // k
0     // null character
```


C++ Strings

Comparing

`==` `!=` `>` `>=` `<` `<=`

The objects are compared character by character lexically.

Can compare:

- `two std::string` objects

- `std::string` object and C-style string literal

- `std::string` object and C-style string variable

C++ Strings

Comparing

```
string s1 {"Apple"};  
string s2 {"Banana"};  
string s3 {"Kiwi"};  
string s4 {"apple"};  
string s5 {s1};           // Apple
```

```
s1 == s5           // True  
s1 == s2           // False  
s1 != s2           // True  
s1 < s2            // True  
s2 > s1            // True  
s4 < s5            // False  
s1 == "Apple";     // True
```

C++ Strings

Substrings – `substr()`

Extracts a substring from a `std::string`

`object.substr(start_index, length)`

```
string s1 {"This is a test"};
```

```
cout << s1.substr(0,4); // This
```

```
cout << s1.substr(5,2); // is
```

```
cout << s1.substr(10,4); // test
```


C++ Strings

Searching – `find()`

Returns the index of a substring in a `std::string`

`object.find(search_string)`

```
string s1 {"This is a test"};

cout << s1.find("This"); // 0
cout << s1.find("is");  // 2
cout << s1.find("test"); // 10
cout << s1.find('e');    // 11
cout << s1.find("is", 4); // 5
cout << s1.find("XX");   // string::npos
```

C++ Strings

Removing characters – `erase()` and `clear()`

Removes a substring of characters from a `std::string`

```
object.erase(start_index, length)
```

```
string s1 {"This is a test"};
```

```
cout << s1.erase(0,5); // is a test
```

```
cout << s1.erase(5,4); // is a
```

```
s1.clear(); // empties string s1
```

C++ Strings

Other useful methods

```
string s1 {"Frank"};
```

```
cout << s1.length() << endl; // 5
```

```
s1 += " James";
```

```
cout << s1 << endl; // Frank James
```

Many more...

C++ Strings

Input >> and getline()

Reading std::string from cin

```
string s1;
cin >> s1;           // Hello there
                      // Only accepts up to the first space
cout << s1 << endl;  // Hello

getline(cin, s1);     // Read entire line until \n
cout << s1 << endl;   // Hello there

getline(cin, s1, 'x'); // this isx
cout << s1 << endl;   // this is
```

Functions

- Function
 - definition
 - prototype
 - Parameters and pass-by-value
 - `return` statement
 - default parameter values
 - overloading
 - passing arrays to function
 - pass-by-reference
 - `inline` functions
 - `auto` return type
 - recursive functions

What is a function?

- C++ programs
 - C++ Standard Libraries (functions and classes)
 - Third-party libraries (functions and classes)
 - Our own functions and classes
- Functions allow the modularization of a program
 - Separate code into logical self-contained units
 - These units can be reused

What is a function?

```
int main() {  
  
    // read input  
    statement1;  
    statement2;  
    statement3;  
    statement4;  
  
    // process input  
    statement5;  
    statement6;  
    statement7;  
  
    // provide output  
    statement8;  
    statement9;  
    statement10;  
  
    return 0;  
}
```

Modularized Code

```
int main() {  
  
    // read input  
    read_input();  
  
    // process input  
    process_input();  
  
    // provide output  
    provide_output();  
  
    return 0;  
}
```

What is a function?

```
int main() {  
  
    read_input();  
  
    process_input();  
  
    provide_output();  
  
    return 0;  
}
```

```
read_input() {  
    statement1;  
    statement2;  
    statement3;  
    statement4;  
}  
  
process_input() {  
    statement5;  
    statement6;  
    statement7;  
}  
  
provide_output() {  
    statement8;  
    statement9;  
    statement10;  
}
```


What is a function?

Boss/Worker analogy

- Write your code to the function specification
 - Understand what the function does
 - Understand what information the function needs
 - Understand what the function returns
 - Understand any errors the function may produce
 - Understand any performance constraints
-
- Don't worry about HOW the function works internally
 - Unless you are the one writing the function!

What is a function?

Example `<cmath>`

- Common mathematical calculations
- Global functions called as:

```
function_name(argument);
```

```
function_name(argument1, argument2, ...);
```

```
cout << sqrt(400.0) << endl;    // 20.0
```

```
double result;
```

```
result = pow(2.0, 3.0);          // 2.0^3.0
```

What is a function?

User-defined functions

- We can define our own functions
- Here is a preview

```
/* This is a function that expects two integers a and b
   It calculates the sum of a and b and returns it to the caller
   Note that we specify that the function returns an int
*/

int add_numbers(int a, int b)
{
    return a + b;
}

// I can call the function and use the value that is returns

cout << add_numbers(20, 40);
```

What is a function?

User-defined functions

- Return zero if any of the arguments are negative

```
/* This is a function that expects two integers a and b
   It calculates the sum of a and b and returns it to the caller
   Only if a or b are non-negative. Otherwise, it returns 0
   Note that we specify that the function returns an int
*/

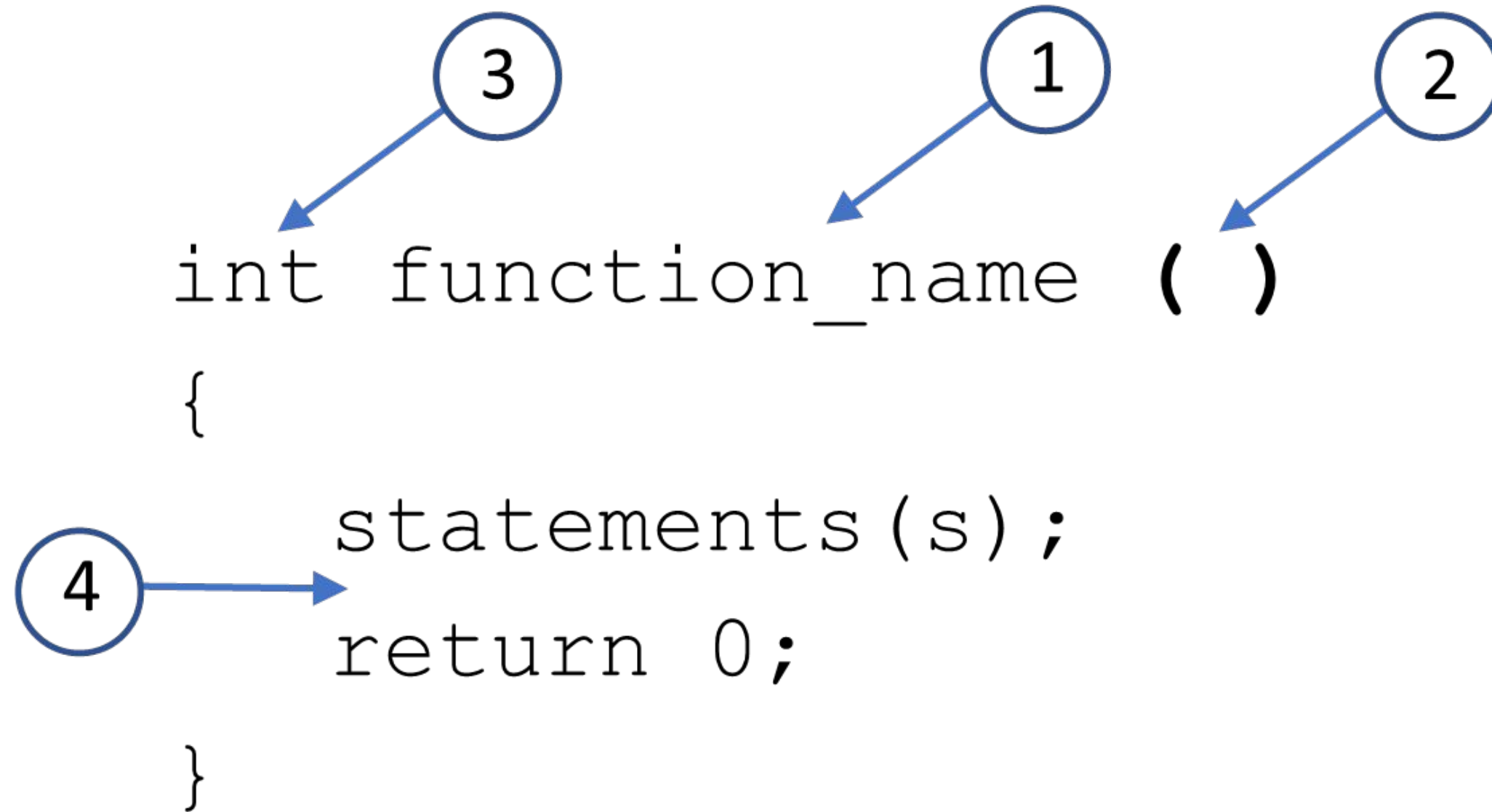
int add_numbers(int a, int b)
{
    if (a < 0 || b < 0)
        return 0;
    else
        return a + b;
}
```

Defining Functions

- name
 - the name of the function
 - same rules as for variables
 - should be meaningful
 - usually a verb or verb phrase
- parameter list
 - the variables passed into the function
 - their types must be specified
- return type
 - the type of the data that is returned from the function
- body
 - the statements that are executed when the function is called
 - in curly braces {}

Defining Functions

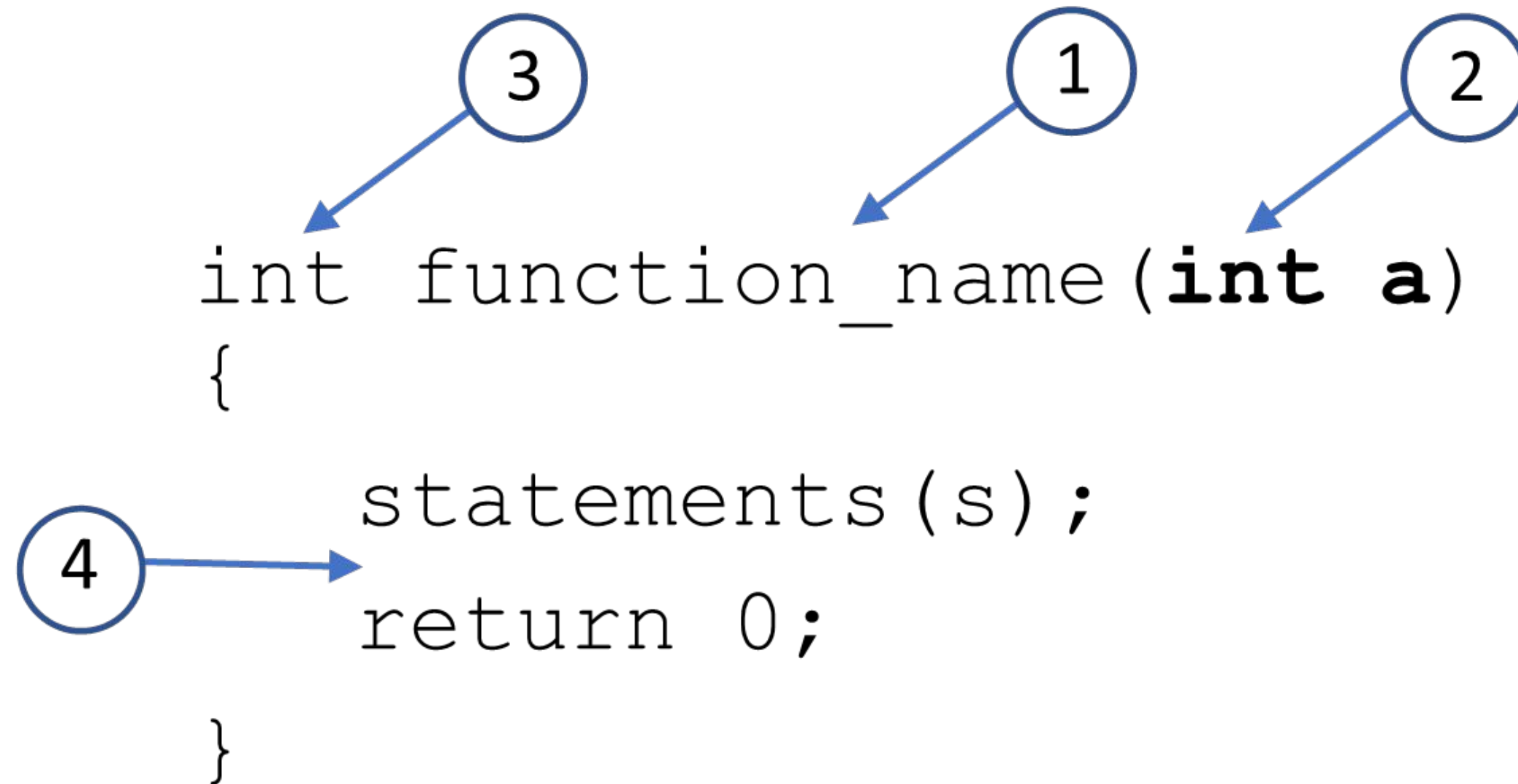
Example with no parameters



1. Name
2. Parameters
3. Return type
4. Body

Defining Functions

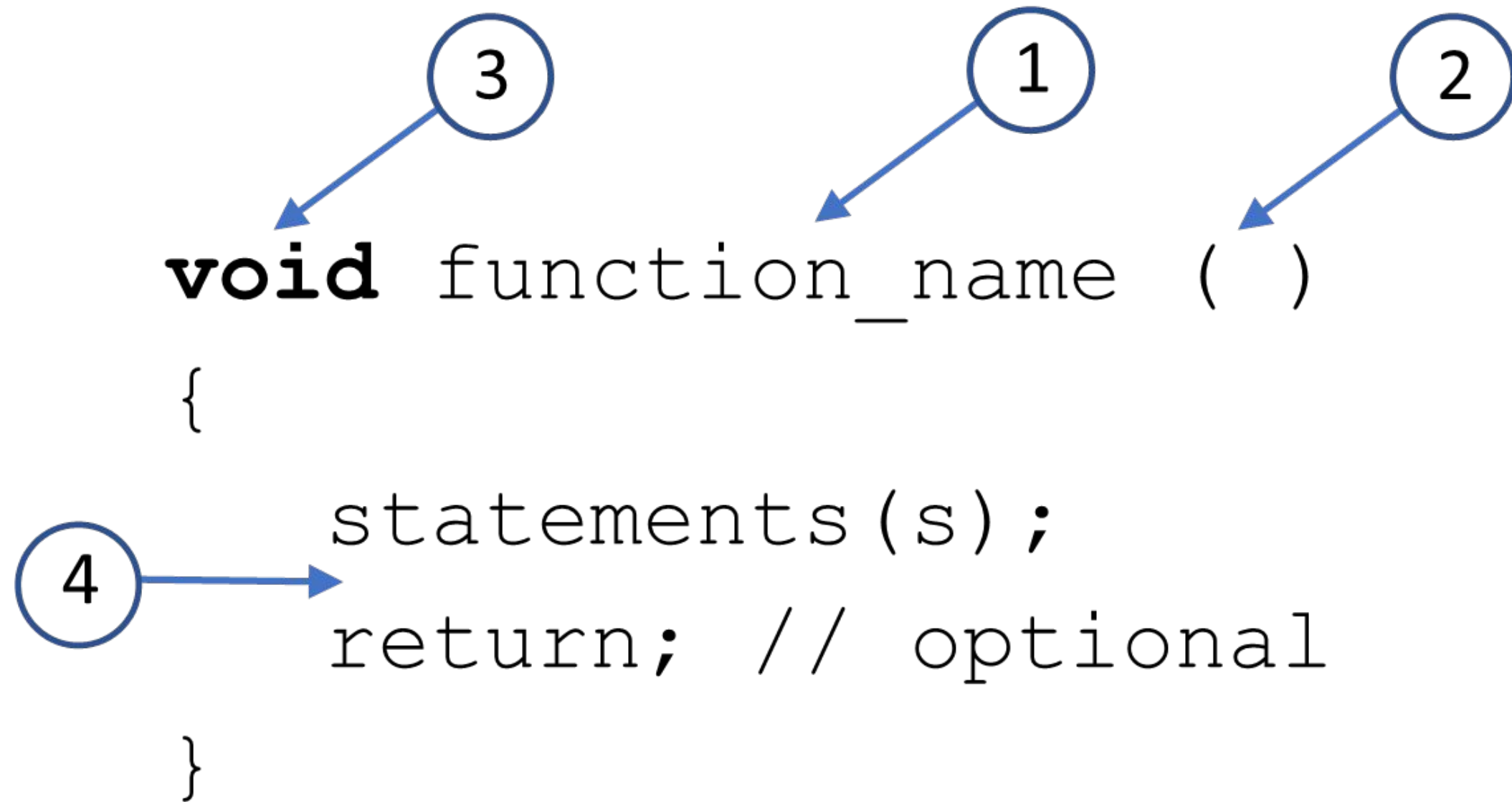
Example with 1 parameter



1. Name
2. Parameters
3. Return type
4. Body

Defining Functions

Example with no return type (void)



1. Name
2. Parameters
3. No return type
4. Body

Defining Functions

Example with multiple parameters

```
void function_name(int a, std::string b)
{
    statements(s);
    return; // optional
}
```

Defining Functions

A function with no return type and no parameters

```
void say_hello () {  
    cout << "Hello" << endl;  
}
```

Calling a function

```
void say_hello () {  
    cout << "Hello" << endl;  
}
```

```
int main() {  
    say_hello();  
    return 0;  
}
```

Calling a function

```
void say_hello () {  
    cout << "Hello" << endl;  
}  
  
int main() {  
    for (int i{1} i<=10; ++i)  
        say_hello();  
    return 0;  
}
```

Calling a function

```
void say_world () {  
    cout << " World" << endl;  
}
```

```
void say_hello () {  
    cout << "Hello" << endl;  
    say_world();  
}
```

```
int main() {  
    say_hello();  
    return 0;  
}
```


Calling a function

```
void say_world () {  
    cout << " World" << endl;  
    cout << " Bye from say_world" << endl;  
}  
  
void say_hello () {  
    cout << "Hello" << endl;  
    say_world();  
    cout << " Bye from say_hello" << endl;  
}  
  
int main() {  
    say_hello();  
    cout << " Bye from main" << endl;  
    return 0;  
}
```

```
Hello  
World  
Bye from say_world  
Bye from say_hello  
Bye from main
```

Calling functions

- Functions can call other functions
- Compiler must know the function details **BEFORE** it is called!

```
int main() {  
    say_hello(); // called BEFORE it is defined ERROR  
    return 0;  
}
```

```
void say_hello ()  
{  
    cout << "Hello" << endl;  
}
```

Function Prototypes

- **The compiler must 'know' about a function before it is used**
 - Define functions before calling them
 - OK for small programs
 - Not a practical solution for larger programs
 - Use function prototypes
 - Tells the compiler what it needs to know without a full function definition
 - Also called forward declarations
 - Placed at the beginning of the program
 - Also used in our own header files (.h) – more about this later

Example

```
int function_name(); // prototype
```

```
int function_name()  
{  
    statements(s);  
    return 0;  
}
```

Example

```
int function_name(int);    // prototype  
                        // or
```

```
int function_name(int a); // prototype
```

```
int function_name(int a) {  
    statements(s);  
    return 0;  
}
```

Example

```
void function_name (); // prototype
```

```
void function_name (  
{  
    statements (s);  
    return; // optional  
}
```

Example

```
void function_name(int a, std::string b);  
// or  
void function_name(int, std::string);
```

```
void function_name(int a, std::string b)  
{  
    statements(s);  
    return; // optional  
}
```

A function with no return type and no parameters

```
void say_hello();
```

```
void say_hello() {  
    cout << "Hello" << endl;  
}
```


Calling a function

```
void say_hello();
```

```
int main() {  
    say_hello();           // OK  
    say_hello(100);        // Error  
    cout << say_hello(); // Error  
                        // No return value  
    return 0;  
}
```

Example

```
void say_hello(); // prototype
void say_world(); // prototype

int main() {
    say_hello();
    cout << " Bye from main" << endl;
    return 0;
}
```

```
void say_world () {
    cout << " World" << endl;
    cout << " Bye from say_world" << endl;
}

void say_hello () {
    cout << "Hello" << endl;
    say_world();
    cout << " Bye from say_hello" << endl;
}
```

Function Parameters

- When we call a function we can pass in data to that function
- In the function call they are called arguments
- In the function definition they are called parameters
- They must match in number, order, and in type

Example

```
int add_numbers(int, int);           // prototype
```

```
int main() {  
    int result {0};  
    result = add_numbers(100,200); // call  
    return 0;  
}
```

```
int add_numbers(int a, int b) { // definition  
    return a + b;  
}
```

Example

```
void say_hello(std::string name) {  
    cout << "Hello " << name << endl;  
}
```

```
say_hello("Frank");
```

```
std::string my_dog {"Buster"};  
say_hello(my_dog);
```

Pass-by-value

- When you pass data into a function it is passed-by-value
- A copy of the data is passed to the function
- Whatever changes you make to the parameter in the function does NOT affect the argument that was passed in.
- Formal vs. Actual parameters
 - Formal parameters – the parameters defined in the function header
 - Actual parameters – the parameter used in the function call, the arguments

Example

```
void param_test(int formal) { // formal is a copy of actual
    cout << formal << endl;    // 50
    formal = 100;               // only changes the local copy
    cout << formal << endl;    // 100
}
```

```
int main() {
    int actual {50};
    cout << actual << endl;    // 50
    param_test(actual);        // pass in 50 to param_test
    cout << actual << endl;    // 50 - did not change
    return 0
}
```

Function Return Statement

- If a function returns a value then it must use a `return` statement that returns a value
- If a function does not return a value (`void`) then the `return` statement is optional
- `return` statement can occur anywhere in the body of the function
- `return` statement immediately exits the function
- We can have multiple `return` statements in a function
 - Avoid many return statements in a function
- The return value is the result of the function call

Default Argument Values

- When a function is called, all arguments must be supplied
- Sometimes some of the arguments have the same values most of the time
- We can tell the compiler to use default values if the arguments are not supplied
- Default values can be in the prototype or definition, not both
 - best practice – in the prototype
 - must appear at the tail end of the parameter list
- Can have multiple default values
 - must appear consecutively at the tail end of the parameter list

Default Argument Values

Example – no default arguments

```
double calc_cost(double base_cost, double tax_rate);
```

```
double calc_cost(double base_cost, double tax_rate) {  
    return base_cost += (base_cost * tax_rate);  
}
```

```
int main() {  
    double cost {0};  
    cost = calc_cost(100.0, 0.06);  
    return 0;  
}
```

Default Argument Values

Example – single default argument

```
double calc_cost(double base_cost, double tax_rate = 0.06);
```

```
double calc_cost(double base_cost, double tax_rate) {  
    return base_cost += (base_cost * tax_rate);  
}
```

```
int main() {  
    double cost {0};  
    cost = calc_cost(200.0);    // will use the default tax  
    cost = calc_cost (100.0, 0.08);    // will use 0.08 not the default  
    return 0;  
}
```


Default Argument Values

Example – multiple default arguments

```
double calc_cost(double base_cost, double tax_rate = 0.06, double shipping = 3.50);
```

```
double calc_cost(double base_cost, double tax_rate, double shipping) {  
    return base_cost += (base_cost * tax_rate) + shipping;  
}
```

```
int main() {  
    double cost {0};  
    cost = calc_cost (100.0, 0.08, 4.25); // will use no defaults  
    cost = calc_cost(100.0, 0.08);      // will use default shipping  
    cost = calc_cost(200.0);           // will use default tax and shipping  
    return 0;  
}
```

Overloading Functions

- We can have functions that have different parameter lists that have the same name
- Abstraction mechanism since we can just think 'print' for example
- A type of polymorphism
 - We can have the same name work with different data types to execute similar behavior
- The compiler must be able to tell the functions apart based on the parameter lists and argument supplied

Overloading Functions

Example

```
int add_numbers(int, int);    // add ints
double add_numbers(double, double); // add doubles

int main() {
    cout << add_numbers(10,20) << endl;    // integer
    cout << add_numbers(10.0, 20.0) << endl; // double
    return 0;
}
```

Overloading Functions

Example

```
int add_numbers(int a, int b) {  
    return a + b;  
}
```

```
double add_numbers(double a, double b) {  
    return a + b;  
}
```

Overloading Functions

Example

```
void display(int n);  
void display(double d);  
void display(std::string s);  
void display(std::string s, std::string t);  
void display(std::vector<int> v);  
void display(std::vector<std::string> v);
```


Overloading Functions

Return type is not considered

```
int    get_value() ;
```

```
double get_value() ;
```

```
// Error
```

```
cout << get_value() << endl; // which one?
```

Passing Arrays To Functions

- We can pass an array to a function by providing square brackets in the formal parameter description

```
void print_array(int numbers []);
```

- The array elements are NOT copied
- Since the array name evaluates to the location of the array in memory – this address is what is copied
- So the function has no idea how many elements are in the array since all it knows is the location of the first element (the name of the array)

Example

```
void print_array(int numbers []);
```

```
int main() {  
    int my_numbers[] {1,2,3,4,5};  
    print_array(my_numbers);  
    return 0;  
}
```

```
void print_array(int numbers []) {  
    // Doesn't know how many elements are in the array???  
    // we need to pass in the size!!  
}
```


Example

```
void print_array(int numbers [], size_t size);

int main() {
    int my_numbers[] {1,2,3,4,5};
    print_array(my_numbers, 5);    / 1 2 3 4 5
    return 0;
}

void print_array(int numbers [], size_t size) {
    for (size_t i{0}; i < size; ++i )
        cout << numbers[i] << endl;
}
```

Example

- Since we are passing the location of the array
 - The function can modify the actual array!

```
void zero_array(int numbers [], size_t size) {  
    for (size_t i{0}; i < size; ++i )  
        numbers[i] = 0;                // zero out array element  
}  
  
int main() {  
    int my_numbers[] {1,2,3,4,5};  
    zero_array(my_numbers, 5);        // my_numbers is now zeroes!  
    print_array(my_numbers, 5);      // 0 0 0 0 0  
    return 0;  
}
```

const parameters

- We can tell the compiler that function parameters are const (read-only)
- This could be useful in the print_array function since it should NOT modify the array

```
void print_array(const int numbers [], size_t size) {  
    for (size_t i{0}; i < size; ++i )  
        cout << numbers[i] << endl;  
    numbers[i] = 0;    // any attempt to modify the array  
                      // will result in a compiler error  
}
```

Pass by Reference

- Sometimes we want to be able to change the actual parameter from within the function body
- In order to achieve this we need the location or address of the actual parameter
- We saw how this is the effect with array, but what about other variable types?
- We can use reference parameters to tell the compiler to pass in a reference to the actual parameter.
- The formal parameter will now be an alias for the actual parameter

Example

```
void scale_number(int &num) ;           // prototype

int main() {
    int number {1000};
    scale_number(number);                // call
    cout << number << endl;             // 100
    return 0;
}

void scale_number(int &num) {           // definition
    if (num > 100)
        num = 100;
}
```


Example

```
void swap(int &a, int &b);
```

```
int main() {  
    int x{10}, y{20};  
    cout << x << " " << y << endl;    // 10 20  
    swap(x, y);  
    cout << x << " " << y << endl;    // 20 10  
    return 0;  
}
```

```
void swap(int &a, int &b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

vector example – pass by value

```
void print(std::vector<int> v);
```

```
int main() {  
    std::vector<int> data {1,2,3,4,5};  
    print(data);           // 1 2 3 4 5  
    return 0;  
}
```

```
void print(std::vector<int> v) {  
    for (auto num: v)  
        cout << num << endl;  
}
```

vector example – pass by reference

```
void print(std::vector<int> &v);
```

```
int main() {  
    std::vector<int> data {1,2,3,4,5};  
    print(data);           // 1 2 3 4 5  
    return 0;  
}
```

```
void print(std::vector<int> &v) {  
    for (auto num: v)  
        cout << num << endl;  
}
```


vector example – pass by const reference

```
void print(const std::vector<int> &v) ;

int main() {
    std::vector<int> data {1,2,3,4,5};
    print(data);                // 1 2 3 4 5
    return 0;
}

void print(const std::vector<int> &v) {
    v.at(0) = 200;              // ERROR
    for (auto num: v)
        cout << num << endl;
}
```

Scope Rules

- C++ uses scope rules to determine where an identifier can be used
- C++ uses static or lexical scoping
- Local or Block scope
- Global scope

Local or Block scope

- Identifiers declared in a block { }
- Function parameters have block scope
- Only visible within the block { } where declared
- Function local variables are only active while the function is executing
- Local variables are NOT preserved between function calls
- With nested blocks inner blocks can 'see' but outer blocks cannot 'see' in

Static local variables

- Declared with static qualifier

```
static int value {10};
```

- Value IS preserved between function calls
- Only initialized the first time the function is called

Global scope

- Identifier declared outside any function or class
- Visible to all parts of the program after the global identifier has been declared
- Global constants are OK
- Best practice – don't use global variables

How do Function Calls Work?

- **Functions use the 'function call stack'**
 - Analogous to a stack of books
 - LIFO – Last In First Out
 - push and pop
- **Stack Frame or Activation Record**
 - Functions must return control to function that called it
 - Each time a function is called we create a new activation record and push it on stack
 - When a function terminates we pop the activation record and return
 - Local variables and function parameters are allocated on the stack
- **Stack size is finite – Stack Overflow**

Inline Functions

- Function calls have a certain amount of overhead
- You saw what happens on the call stack
- Sometimes we have simple functions
- We can **suggest** to the compiler to compile them 'inline'
 - avoid function call overhead
 - generate inline assembly code
 - faster
 - could cause code bloat
- Compilers optimizations are very sophisticated
 - will likely inline even without your suggestion

Example

```
inline int add_numbers(int a, int b) { // definition
    return a + b;
}
```

```
int main() {
    int result {0};
    result = add_numbers(100, 200); // call
    return 0;
}
```


Recursive Functions

- A recursive function is a function that calls itself
 - Either directly or indirectly through another function
- Recursive problem solving
 - Base case
 - Divide the rest of problem into subproblem and do recursive call
- There are many problems that lend themselves to recursive solutions
- Mathematic – factorial, Fibonacci, fractals,...
- Searching and sorting – binary search, search trees, ...

Example - Factorial

$$\begin{array}{l} 0! = 1 \\ n! = n * (n-1)! \end{array}$$

- **Base case:**
 - $\text{factorial}(0) = 1$
- **Recursive case:**
 - $\text{factorial}(n) = n * \text{factorial}(n-1)$

Example - Factorial

```
unsigned long long factorial(unsigned long long n) {  
    if (n == 0)  
        return 1;                // base case  
    return n * factorial(n-1);    // recursive case  
}  
  
int main() {  
    cout << factorial(8) << endl; // 40320  
    return 0;  
}
```

Example - Fibonacci

```
Fib (0)  =  0
Fib (1)  =  1
Fib (n)  =  Fib (n-1)  +  Fib (n-2)
```

- **Base case:**
 - $\text{Fib}(0) = 0$
 - $\text{Fib}(1) = 1$
- **Recursive case:**
 - $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

Example - Factorial

```
unsigned long long fibonacci (unsigned long long n) {  
    if (n <= 1)  
        return n;                // base cases  
    return fibonacci (n-1) + fibonacci (n-2); // recursion  
}  
  
int main() {  
    cout << fibonacci (30) << endl; // 832040  
    return 0;  
}
```


Important notes

- If recursion doesn't eventually stop you will have infinite recursion
- Recursion can be resource intensive
- Remember the base case(s)
 - It terminates the recursion
- Only use recursive solutions when it makes sense
- Anything that can be done recursively can be done iteratively
 - Stack overflow error

Overview

- What is a pointer?
- Declaring pointers
- Storing addresses in pointers
- Dereferencing pointers
- Dynamic memory allocation
- Pointer arithmetic
- Pointers and arrays
- Pass-by-reference with pointers
- const and pointers
- Using pointers to functions
- Potential pointer pitfalls
- What is a reference?
- Review passing references to functions
- const and references
- Reference variables in range-based for loops
- Potential reference pitfalls
- Raw vs. Smart pointers

What is a Pointer?

- A variable
 - whose value is an address
- What can be at that address?
 - Another variable
 - A function
- Pointers point to variables or functions?
- If x is an integer variable and its value is 10 then I can declare a pointer that points to it
- To use the data that the pointer is pointing to you must know its type

Why use Pointers?

Can't I just use the variable or function itself?

Yes, but not always

- Inside functions, pointers can be used to access data that are defined outside the function. Those variables may not be in scope so you can't access them by their name
- Pointers can be used to operate on arrays very efficiently
- We can allocate memory dynamically on the heap or free store.
 - This memory doesn't even have a variable name.
 - The only way to get to it is via a pointer
- With OO. pointers are how polymorphism works!
- Can access specific addresses in memory
 - useful in embedded and systems applications

Declaring Pointers

```
variable_type *pointer_name;
```

```
int *int_ptr;  
double* double_ptr;  
char *char_ptr;  
string *string_ptr;
```

Declaring Pointers

Initializing pointer variables to 'point nowhere'

```
variable_type *pointer_name {nullptr};
```

```
int *int_ptr {};  
double* double_ptr {nullptr};  
char *char_ptr {nullptr};  
string *string_ptr {nullptr};
```

Declaring Pointers

Initializing pointer variables to 'point nowhere'

- Always initialize pointers
- Uninitialized pointers contain garbage data and can 'point anywhere'
- Initializing to zero or `nullptr` (C++ 11) represents address zero
 - implies that the pointer is 'pointing nowhere'
- If you don't initialize a pointer to point to a variable or function then you should initialize it to `nullptr` to 'make it null'

Accessing Pointer Address?

& the address operator

- Variables are stored in unique addresses
- Unary operator
- Evaluates to the address of its operand
 - Operand cannot be a constant or expression that evaluates to temp values

```
int num{10};
```

```
cout << "Value of num is: " << num << endl;    // 10
```

```
cout << "sizeof of num is: " << sizeof num << endl;    // 4
```

```
cout << "Address of num is: " << &num << endl;    // 0x61ff1c
```


Accessing Pointer Address?

& the address operator - example

```
int *p;

cout << "Value of p is: " << p << endl; // 0x61ff60 - garbage

cout << "Address of p is: " << &p << endl; // 0x61ff18

cout << "sizeof of p is: " << sizeof p << endl; // 4

p = nullptr; // set p to point nowhere

cout << "Value of p is: " << p << endl; // 0
```

Accessing Pointer Addresses?

`sizeof` a pointer variable

- Don't confuse the size of a pointer and the size of what it points to
- All pointers in a program have the same size
- They may be pointing to very large or very small types

```
int *p1 {nullptr};  
double *p2 {nullptr};  
unsigned long long *p3 {nullptr};  
vector<string> *p4 {nullptr};  
string *p5 {nullptr};
```


Storing an Address in Pointer Variable?

Typed pointers

- The compiler will make sure that the address stored in a pointer variable is of the correct type

```
int score {10};
```

```
double high_temp {100.7};
```

```
int *score_ptr {nullptr};
```

```
score_ptr = &score;    // OK
```

```
score_ptr = &high_temp; // Compiler Error
```


Storing an Address in Pointer Variable?

& the address operator

- Pointers are variables so they can change
- Pointers can be null
- Pointers can be uninitialized

```
double high_temp {100.7};  
double low_temp {37.2};
```

```
double *temp_ptr;
```

```
temp_ptr = &high_temp; // points to high_temp  
temp_ptr = &low_temp;  // now points to low_temp
```

```
temp_ptr = nullptr;
```

Dereferencing a Pointer

Access the data we're pointing to – dereferencing a pointer

- If `score_ptr` is a pointer and has a valid address
- Then you can access the data at the address contained in the `score_ptr` using the dereferencing operator `*`

```
int score {100};  
int *score_ptr {&score};  
  
cout << *score_ptr << endl;    // 100  
  
*score_ptr = 200;  
cout << *score_ptr << endl;    // 200  
cout << score << endl;         // 200
```

Dereferencing a Pointer

Access the data we're pointing to

```
double high_temp {100.7};  
double low_temp {37.4};  
double *temp_ptr {&high_temp};  
  
cout << *temp_ptr << endl;    // 100.7  
  
temp_ptr = &low_temp;  
  
cout << *temp_ptr << endl;    // 37.4
```

Dereferencing a Pointer

Access the data we're pointing to

```
string name {"Frank"};
```

```
string *string_ptr {&name};
```

```
cout << *string_ptr << endl;    // Frank
```

```
name = "James";
```

```
cout << *string_ptr << endl;    // James
```

Dynamic Memory Allocation

Allocating storage from the heap at runtime

- We often don't know how much storage we need until we need it
- We can allocate storage for a variable at run time
- Recall C++ arrays
 - We had to explicitly provide the size and it was fixed
 - But vectors grow and shrink dynamically
- We can use pointers to access newly allocated heap storage

Dynamic Memory Allocation

using new to allocate storage

```
int *int_ptr {nullptr};

int_ptr = new int;           // allocate an integer on the heap

cout << int_ptr << endl;     // 0x2747f28

cout << *int_ptr << endl;    // 41188048 - garbage

*int_ptr = 100;

cout << *int_ptr << endl;    // 100
```

Dynamic Memory Allocation

using delete to deallocate storage

```
int *int_ptr {nullptr};
```

```
int_ptr = new int;    // allocate an integer on the heap
```

```
. . .
```

```
delete int_ptr;    // frees the allocated storage
```

Dynamic Memory Allocation

using `new[]` to allocate storage for an array

```
int *array_ptr {nullptr};  
int size {};
```

```
cout << "How big do you want the array? ";  
cin >> size;
```

```
array_ptr = new int[size]; // allocate array on the heap
```

```
// We can access the array here  
// we'll see how in a few slides
```


Dynamic Memory Allocation

using `delete []` to deallocate storage for an array

```
int *array_ptr {nullptr};  
int size {};
```

```
cout << "How big do you want the array?";  
cin >> size;
```

```
array_ptr = new int[size]; // allocate array on the heap
```

```
. . .
```

```
delete [] array_ptr; // free allocated storage
```

Relationship Between Arrays and Pointers

- The value of an array name is the address of the first element in the array
- The value of a pointer variable is an address
- If the pointer points to the same data type as the array element then the pointer and array name can be used interchangeably (almost)

Relationship Between Arrays and Pointers

```
int scores[] {100, 95, 89};

cout << scores << endl;      // 0x61fec8
cout << *scores << endl;    // 100

int *score_ptr {scores};

cout << score_ptr << endl;  // 0x61fec8

cout << *score_ptr << endl; // 100
```

Relationship Between Arrays and Pointers

```
int scores[] {100, 95, 89};  
  
int *score_ptr {scores};  
  
cout << score_ptr[0] << endl; // 100  
  
cout << score_ptr[1] << endl; // 95  
  
cout << score_ptr[2] << endl; // 89
```

Using pointers in expressions

```
int scores[] {100, 95, 89};
```

```
int *score_ptr {scores};
```

```
cout << score_ptr << endl;      // 0x61ff10
```

```
cout << (score_ptr + 1) << endl; // 0x61ff14
```

```
cout << (score_ptr + 2) << endl; // 0x61ff18
```


Using pointers in expressions

```
int scores[] {100, 95, 89};
```

```
int *score_ptr {scores};
```

```
cout << *score_ptr << endl;    // 100
```

```
cout << *(score_ptr + 1) << endl; // 95
```

```
cout << *(score_ptr + 2) << endl; // 89
```

Subscript and Offset notation equivalence

```
int array_name[] {1,2,3,4,5};  
  
int *pointer_name {array_name};
```

Subscript Notation	Offset Notation
<code>array_name[index]</code>	<code>* (array_name + index)</code>
<code>pointer_name[index]</code>	<code>* (pointer_name + index)</code>

Pointer Arithmetic

- Pointers can be used in
 - Assignment expressions
 - Arithmetic expressions
 - Comparison expressions
- C++ allows pointer arithmetic
- Pointer arithmetic only makes sense with raw arrays